



**2026 Future Computing Summer Internship
at the
Laboratory for Physical Sciences (LPS)**

Collect & Adjust Server-Level Power Consumption

Emily Doyle
Alexander Johnson
Daniel Lee

August 3, 2026



Contents

Introduction.....	1
Background.....	1
RAPL.....	2
Overview.....	2
Sampling rate and false spikes.....	2
Power capping (PL1 / PL2 / tau).....	3
RAPL vs. Redfish.....	4
Portability.....	4
Monitoring Live Server Activity.....	5
The Stack.....	5
Evolution of Data Collection.....	5
Cost of Observation.....	5
Validation.....	7
Model.....	8
The Importance of Timing.....	8
Regression Baseline.....	8
Heuristics.....	8
Classification Models.....	9
Random Forest.....	10
Prediction to Detection.....	11
MATLAB.....	13
Early Motivations and Layout.....	13
Simulink.....	13
Metrics.....	15
Solutions to Smoothing Power.....	16
Overview.....	16
1. Ramp.c.....	16
2. Ramp Down.....	18
3. Governor.....	25
Experiment Design.....	26
Server to MATLAB Pipeline.....	26
Workloads.....	28
HPL (ai_load.sh).....	28
AI Sim 2 Workload.....	29
Step Workload.....	30
Metrics.....	31
Results.....	32
First Glance.....	32
Server Level Impact.....	35
Grid Level Impact.....	36
Cost Analysis.....	37
Reproducibility of Data.....	37
Complete Dataset.....	37
Scoring Detector.....	38
Conclusion.....	38
AI Acknowledgement.....	39
References.....	40

Introduction

Over the past several years, advances in high-performance computing (HPC) have resulted in the proliferation of bulk-synchronous jobs. By nature, these workloads experience large fluctuations in their power consumption, potentially threatening the stability of the grid. Though hardware solutions to this problem have been developed, they may not be available in all situations, leading this paper to explore software solutions.

At the beginning of this endeavor, a comprehensive software stack was created to measure the power consumption of a testing server. Once the measurement system was established, three candidate solutions were developed. In order to evaluate their performance, several workloads were designed, enabling each solution to be tested across various operating conditions. To better understand the significance of these tests, all power profiles were fed into a MATLAB grid simulation. With this, grid instability metrics could be inferred for a full-scale HPC center. Ultimately, one of the three candidate solutions proved to be effective.

Background

In recent years, the development of artificial intelligence has transformed the power signature of data centers. While traditional data centers are characterized by the concurrent operation of numerous independent workloads, AI infrastructure must handle the distinctive qualities of bulk-synchronous jobs. As their name suggests, these loads contain a great number of nodes whose operation must be synchronized.

To comprehend the ramifications of such a scheme, one must understand that bulk-synchronous loads consist of two alternating phases: computation and communication. During the computation phase, each node is responsible for processing a particular collection of data. Once all calculations have concluded, the results must be aggregated through a period of inter-node communication.

Within the context of AI training, the computation phase contains two primary procedures. First, nodes utilize the shared model parameters to generate predictions from their designated training data. Then, each node evaluates its predictions, computing the local gradient of a loss function with respect to model parameters. After these local gradients are obtained, the system enters the communication phase, wherein the gradients are combined to update the shared model parameters.

Looking at the bigger picture, one may question how such organization might alter the power consumption profile of a data center. The answer lies in a simple fact: the computation phase demands significantly more power than the communication phase. When a great number of synchronized nodes alternate between the phases, the disparity is compounded, creating massive fluctuations in power consumption.

If the grid experiences significant oscillations in demand, it is vulnerable to various forms of instability, including harmonic distortion, frequency drift, and voltage flutter. Furthermore, particular oscillation frequencies have the potential to induce resonance within generation

equipment, amplifying the rate of fatigue. Understanding these risks, engineers have developed low-loss energy storage devices, which are capable of absorbing the aforementioned fluctuations. However, it is important to note that such hardware solutions require notable monetary investment, leading many to explore possible software solutions. In this paper, the implementation and performance of various such approaches are investigated.

RAPL

Overview

Intel's Running Average Power Limit (RAPL) is a hardware power-management architecture that maintains a running estimate of consumed energy across several processor domains: the CPU package, cores (PP0), uncore/graphics (PP1), and DRAM. It uses Model Specific Registers (MSRs) to record energy in a counter up to once per millisecond. We calculate power as a running average by dividing the RAPL energy (in microjoules) between two reads by the wall-clock time that has elapsed.

Rather than directly reading from the MSRs, we used the Linux powercap sysfs interface for portability. We note that using powercap requires root access. The interface displays each RAPL domain as a directory in the form `/sys/class/powercap/intel-rapl:<pkg>[:<subzone>]/`, where `<pkg>` is a CPU socket and `<subzone>` can specify the `energy_uj` counter, name (e.g., `package-0`), and `max_energy_range_uj`. Since a modern server is multi-socket, server energy is the sum of all package domains. Using modulo, the energy counter reports deltas between reads from 0 to `max_energy_range_uj`, which is ≈ 262 kJ per package on Mycroft, an HPE server with Xeon CPUs. This ensures positive values by adding `max_energy_range_uj` if we get a negative read.

Sampling rate and false spikes

The literature states that transient power swings occur at the microsecond level, suggesting that teams should pursue the hardware limit for high-frequency power monitoring. Although RAPL can read the energy counter at 1 kHz, we experimentally observed that sampling faster than 10 Hz on Mycroft produced phantom power spikes. We know these kilowatt-order spikes are not real because Mycroft's power ceiling is 410 W. The false spikes occur because, at high rates, the energy counter does not advance on every read, allowing a series of unchanged reads followed by one read that captures all the accumulated energy. Because that delta is wrongly attributed to a single sample interval rather than the total duration of those reads, RAPL incorrectly reports a power spike. We fixed this by only reporting a power value when the counter advanced, dividing the energy delta by the time since the last advance. For Mycroft, we found that sampling above 10 Hz does not yield additional physical information.

Table 1. RAPL powercap constraints

sysfs file	Name	Meaning	Mycroft
constraint_0_power_limit_uw	PL1 (long_term)	Sustained ceiling	205 W
constraint_0_time_window_us	PL1 window	Averaging window for PL1	999424 μ s ($\sim$ 1.0 s)
constraint_1_power_limit_uw	PL2 (short_term)	Burst ceiling	246 W
constraint_1_time_window_us	tau	Burst duration before clamp to PL1	999424 μ s registered; $\sim$ 1.6 s observed

Power capping (PL1 / PL2 / tau)

RAPL can also enforce power limits through the constraints in the table below. The hardware requires $PL2 \geq PL1$. On Mycroft, the CPU defaults are $PL1 = 205$ W and $PL2 = 246$ W, and changing these limits requires root. We did not extensively test different caps, but confirmed RAPL can curb power. The Mycroft CPU maintains PL2 for a measured tau of $\sim$ 1.6 s, then steps down to the $\sim$ 205 W sustained floor. We exploited and worked around this clamp behavior to design power ramping mechanisms.

Table 2. powercap Constraints

sysfs file	Name	Meaning	Mycroft
constraint_0_power_limit_uw	PL1 (long_term)	Sustained ceiling	205 W
constraint_0_time_window_us	PL1 window	Averaging window for PL1	—
constraint_1_power_limit_uw	PL2 (short_term)	Burst ceiling	246 W
constraint_1_time_window_us	tau	Burst duration before clamp to PL1	$\sim$ 1.6 s measured

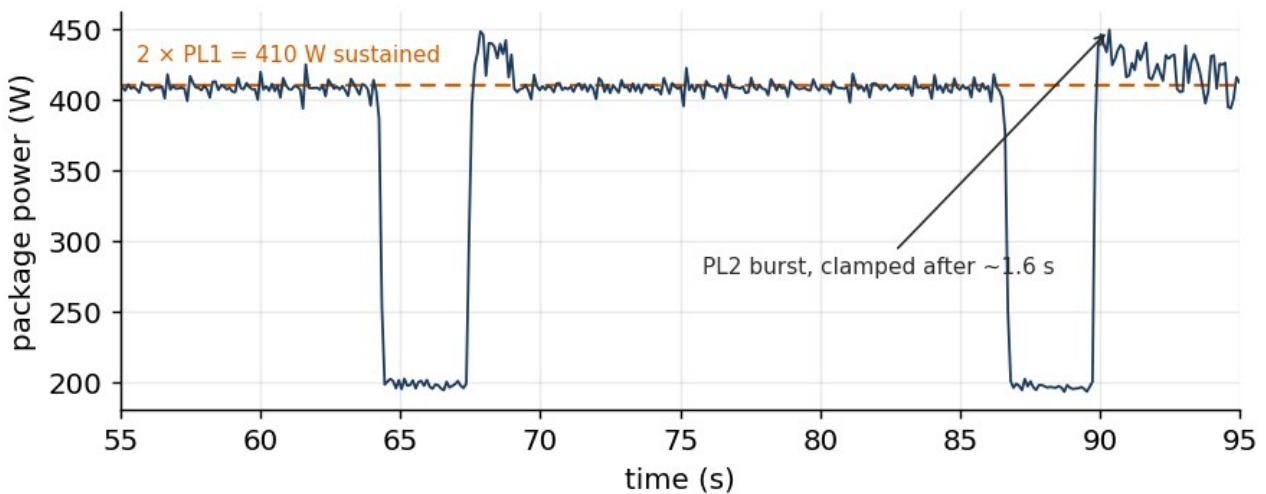


Figure 1.

RAPL vs. Redfish

Originally, our project intended to use Distributed Management Task Force (DMTF) Redfish to manage hardware health and power. Although Redfish is the industry standard, it can only poll power up to once per second. We chose RAPL over Redfish as our primary power telemetry source to prioritize a higher sampling rate and domain granularity. A table comparing RAPL and Redfish is provided below.

Table 3. Telemetry Comparison

	RAPL	Redfish
Scope	CPU package, cores, DRAM	Whole chassis
Granularity	Per-socket, per-domain	Chassis total
Useful rate	~10 Hz	~1 Hz
Path	In-band, host CPU	Out-of-band, BMC
Host overhead	<3% of one core	None
Requires	Root on host	BMC credentials + network
Survives host hang	No	Yes
Capping	Yes — millisecond-window PL1/PL2	Yes — coarse, slow (DCMI)
Portability	x86, vendor-specific	Vendor-neutral standard

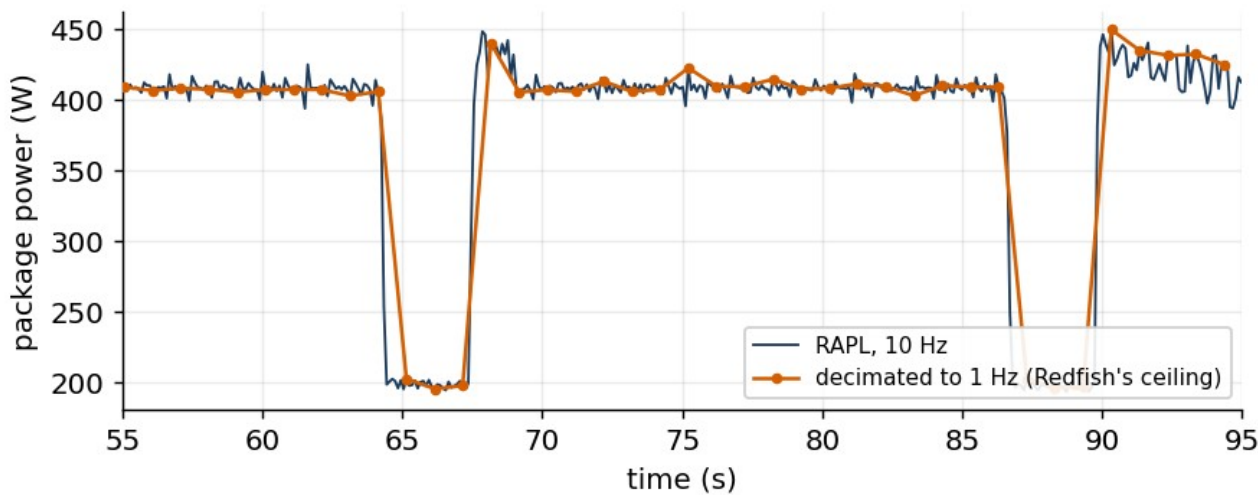


Figure 2.

Because Redfish reports power for the whole chassis, we integrated Redfish to collect system-wide power and thermal management data. Additionally, it supports out-of-band management. Because it is located on the Baseboard Management Controller (BMC), a separate processor on the motherboard, it imposes less CPU overhead than RAPL, which uses the host CPU to poll and forward samples. The Linux top (table of processes) command measured that overhead at under 3% CPU, which we deemed acceptable.

Portability

As mentioned before, the powercap interface is compatible with AMD's RAPL registers. The Windows operating system (OS) uses the same energy-counter MSRs but cannot cap power. Thus, one could read power on AMD hardware and cap power on machines using the Linux OS. If we had more time, we wanted to port the RAPL scripts to Ramius, an HPC cluster that runs on AMD hardware, to compare cap behavior and overshoot dynamics.

Monitoring Live Server Activity

The Stack

To monitor Mycroft's live CPU power over time, we wrote a daemon to poll RAPL and compute mean power at a fixed rate. Our code streams data from RAPL into an InfluxDB time-series database, which Grafana queries for visualization. On Grafana, we built dashboards for CPU power, clock frequency, utilization, and temperature. We also had dashboards sourced from daemons for Linux telemetry, Redfish BMC, and the Power Distribution Unit (PDU), which we will elaborate on below. We used an API token to authenticate writes to the database. InfluxDB and Grafana are maintained on Mycroft as background daemons. Both services are locally hosted, where Grafana serves dashboards on port 3000, and InfluxDB exposes its write and query API on port 8086. Hosting on Mycroft as opposed to another server allowed all project contributors to access our monitoring tools. Mycroft CPU power is sampled at 10 Hz while system, Redfish, and PDU daemons are capped by hardware constraints at 1 Hz, 0.1 Hz, and 1.8 Hz, respectively. All Grafana panels were set to auto-refresh every 5 seconds, allowing us to monitor a variety of metrics quickly.

Evolution of Data Collection

Initially, we collected time-series CSVs and plotted them offline to generate power plots. As we shifted to live data collection, we used the daemons mentioned above. Along with RAPL, we polled the Redfish BMC and the PDU connected to Mycroft for BMC- and PDU- power, frequency, Power Supply Unit (PSU) voltage, temperature, and cooling power. Then, we used Linux system telemetry for disk read/write activity (operations, bytes, throughput), memory, load averages, context switches per second, and run-queue depth (number of runnable processes). Later, we ported some daemons from Python to C for lower Mycroft overhead.

Cost of Observation

When monitoring power, we aimed for minimal operational overhead to collect cleaner data from running workloads and reduce the impact of confounding variables. Because the goal was to study server power, we measured the cost of our stack to account for it. We found that the Python RAPL script used 2.34% of one core and 50.4 MiB. The C script, which collected RAPL per-core statistics and 64 temperature sensors, used only 0.38% of a core and 2.3 MiB. This lowered the

standard deviation of idle package power from 1.84 W with Python to 1.68 W using C, as shown in the table below. The standard deviation for C was the same as when nothing was running, implying that C imposed minimal overhead.

Table 4. Sampler-induced idle noise (120 s @ 10 Hz)

Condition	Mean	Stdev	Min	Max
Nothing Running	199.60 W	1.68 W	194.89 W	204.99 W
rapl_hf_sampler (C), 10 Hz	199.69 W	1.68 W	195.20 W	205.82 W
influx2.py (Python), 10 Hz	199.92 W	1.84 W	194.55 W	205.92 W

We also analyzed our full monitoring stack using 60 s windows for an idle Mycroft. Altogether, our collection tools cost 2.19% of one core, or 0.017% of the server’s capacity, and 443 MiB. Our full analysis is summarized in the table below.

Table 5. Telemetry stack overhead (Mycroft, idle, 60 s windows, 2026-07-22)

Process	Role	Poll Rate	CPU % (1 core)	RSS	Δ Power
rapl_hf_sampler (C)	RAPL + per-core stats + 64 temp sensors	10 Hz	0.38%	2.3 MiB	+0.53 W*
influx2.py (Python predecessor)	same, before the C port	10 Hz	2.34%	50.4 MiB	+0.48 W*
collect_rapl.py (Python)	RAPL package power only	10 Hz	0.30%	8.8 MiB	−0.28 W*
Redfish poller	BMC out-of-band telemetry (69 fields)	0.1 Hz (10 s)	0.43%	56.1 MiB	not measured
PDU poller	Rack PDU real power over HTTP	~1.8 Hz (round-trip bound)	0.47%	26.2 MiB	not measured
Grafana 13.0.2	Dashboards	idle, no client	0.91%	357.7 MiB	−0.57 W*
Collection layer total	C sampler + Redfish + PDU + Grafana		2.19% of one core (0.017% of 128 threads)	~443 MiB	below noise floor
InfluxDB (storage layer)	Time-series database, no writers	—	6.0% settled, ~190% during compaction	396 MiB settled, 1.2 GiB peak	+6.00 W

Mycroft specs: *Dual Intel Xeon Gold 6338, 2 × 32 cores / 128 threads.*

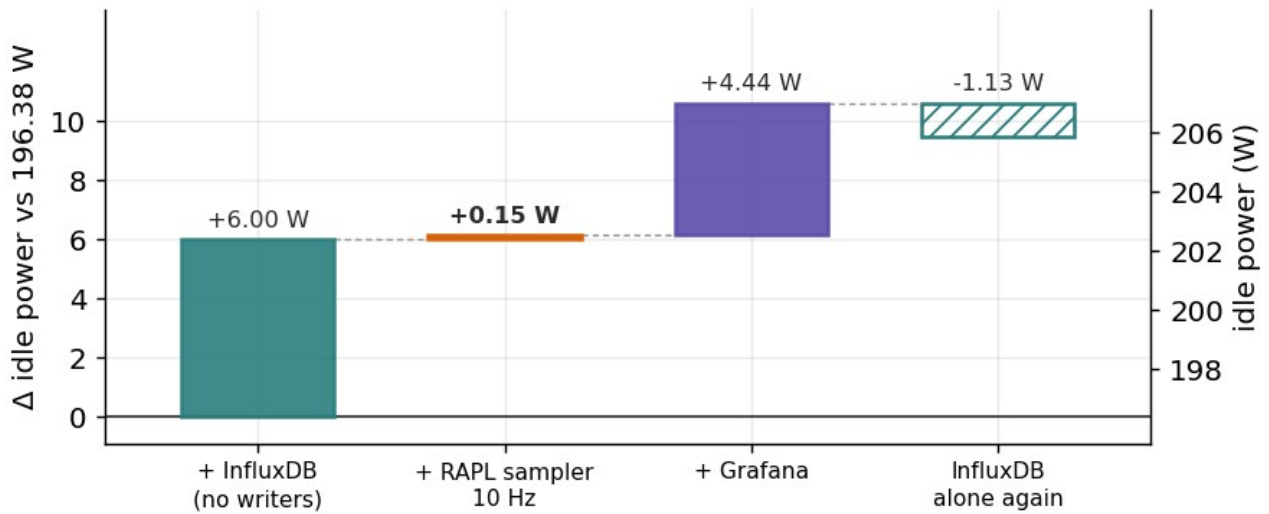
Idle baseline 199.4–200.2 W; baseline-to-baseline drift 0.02 W over nine minutes.
 * Noise floor is ± 0.6 W to account for Grafana measuring below baseline

Our storage layer, InfluxDB, raises the idle CPU power by 6.0 W even without writers, which is more than the entire collection layer costs. In the worst case, the full stack adds 9.5 W. By contrast, our 10 Hz RAPL writer costs 0.15 W and 0.41% of one core. From the data, we can infer that the cost of using in-band telemetry is more impacted by storage than polling frequency. Our table analyzes the power and memory cost for different tools on Mycroft.

Table 6. InfluxDB Cost by stack layer (Mycroft, idle, 60 s windows, cumulative)

Layer	Baseline	Δ	CPU	RSS	Layer numbers
1. Nothing running	196.38 W	—	stopped	—	—
2. InfluxDB running, no writers	202.38 W	+6.00 W	53.7%	615 MiB	—
3. + RAPL sampler writing 10 Hz	202.53 W	+0.15 W	51.6%	553 MiB	0.41% / 2.5 MiB
4. + Grafana	206.97 W	+4.44 W	184.2%	1168 MiB	0.73% / 361 MiB
5. InfluxDB alone again	205.84 W	+9.46 W vs. 1	188.9%	1200 MiB	—
InfluxDB settled (later observation)	—	—	6.0%	396 MiB	—

PDU and Redfish pollers ran throughout all five phases and are therefore constant in every row.

**Figure 3.**

Validation

We cross-checked our dashboards against system-wide telemetry using `top` and `lscpu`. To further stress-test our panels, we ran various intensive workloads on Mycroft. One primary benchmark we developed and used was a synthetic AI-training load on top of Linpack (HPL), which repeatedly drove all 64 physical CPU cores to full power for ~18 seconds, then dropped to idle for ~3 seconds. This checkpoint-dip cycle resembles the power profiles of data centers dedicated to AI training that we have seen in the literature. From our visualization dashboards, we observed several relationships between data, particularly the close dynamic between CPU utilization and package power. For more rigorous analysis, we employed statistical methods and machine-learning models rather than reading Grafana panels side by side.

Model

The Importance of Timing

A common approach in the literature to the power swing problem is to deploy secondary workloads when the primary job is finished. The secondary workload can be anything that burns power to prevent large drops or upward spikes, which we know damage hardware. While designing a good mechanism to smooth abrupt changes in power is crucial, knowing when to trigger the secondary workload is more important. If we wait too long, the power drop has already occurred. If we execute when the primary is still on, we make the ceiling even higher and waste power. Thus, we aimed to forecast power spikes before they occurred using statistical and machine learning (ML) techniques.

Regression Baseline

Initially, we framed this as a regression problem by using Ordinary Least Squares (OLS) to predict Mycroft’s power over time. The OLS model analyzed the past 3 seconds of power and usage data to calculate the power 5 seconds ahead. Because this technique minimizes global error, our model was terrible at anticipating rare events like power spikes. The metrics we used to quantify performance included precision, recall, precision-recall area under the curve (PR-AUC), and lead time. The table below defines these statistics, along with how the OLS scored on them. Overall, PR-AUC was our main metric for evaluating performance in our models because it accounted for both false positive and false negatives. For that reason, it is the standard for binary classification in ML.

Table 7. Evaluation metrics and the OLS baseline

Data: 192,527 samples at 1 Hz (~53 h). Forecast horizon 5 s, rolling 10-minute training window

Metric	Definition	Relevance Here	OLS score
Precision	Of the spikes we flagged, how many were real?	False alarms burn power for nothing.	0.1145
Recall	Of real spikes, how many did we flag?	Missing spikes means grid damage.	0.0110
PR-AUC	Precision and recall as one number	Spikes are rare, so we need to limit false alarms (precision) and detection failures (recall).	0.1149
Lead time	How far ahead of the spike did we flag?	The secondary workload needs time to ramp up.	2.0 s (median)
Spikes caught	Absolute count.	Intuitive number	19 / 1,735

Heuristics

Because we care about whether abrupt changes in power occur rather than what the power is at a given time in the future, we reframed our approach to classification. Our new goal was to compute

the probability of a spike, a binary occurrence, rather than to forecast a continuous power value. Using several days of InfluxDB data, we ran Analysis of Variance (ANOVA) and tests to see whether anything we were collecting behaved differently before a spike compared to steady operation. We hoped this would reveal indicators of power spikes, which we quantified using either a 99th percentile power derivative (25 W/s) or an Exponentially Weighted Moving Average (EWMA) $|z| > 3$, by statistics convention.

Our heuristics search identified four statistically significant candidates:

1. Rolling server-power coefficient of variation (σ/μ)
2. Rolling server-power peak-to-mean ratio ($\max/\mu$)
3. Server-power EWMA z-score
4. Server-usage slope

We screened each candidate with t-tests and Cohen’s d over a sample size of 170,000 to determine which had the most impact. The analysis deemed the first two metrics large enough in effect size. The table below contains definitions and our full data.

Table 8. Candidate precursors: significance vs. effect size

Candidate feature	What it measures	Significant ($p < 0.05$)?	Cohen’s d	Effect	Kept?
Rolling power coefficient of variation (σ/μ)	Relative volatility over a trailing 30 s window	Yes	1.1–1.2	Large	Yes
Rolling power peak-to-mean ratio ($\max/\mu$)	Burstiness over the same window	Yes	1.1–1.2	Large	Yes
Power EWMA z-score	Deviation from a smoothed running mean	Yes	0.09–0.15	Small	No
Server usage slope	Rate of change of CPU utilization	Yes	0.09–0.15	Small	No

Using both heuristics, we achieved a PR-AUC of 0.54. However, we note that our definition of a spike at the time was flawed because percentile data was largely biased by an idle server, meaning the threshold for what counted as a spike was too low. We addressed this by applying more advanced machine learning techniques to the legacy spike definition.

Classification Models

We used two classic machine learning algorithms for classifying power spikes: logistic regression (LR) and random forest (RF). We found that RF scored higher, with a PR-AUC of 0.68. We believe this is because RF uses many decision trees trained on different random samples of data, allowing for a model with less bias and overfitting than LR. Also, RF is a strong candidate because the decision trees can train in parallel, allowing it to cost less training time than other tree-based approaches like gradient boosting, which trains in serial.

We also considered using other popular techniques. Our KalmanNet-lite attempt failed because there wasn’t a temporal structure in server power profiles, discouraging us from trying Graph

Neural Networks (GNNs) and Convolutional Neural Networks (CNNs). Furthermore, clustering did not make sense because it found operating regimes rather than employing supervised learning to identify spikes.

Random Forest

We wrote an RF daemon on Mycroft to compute a live spike probability and risk threshold, where the model would predict a spike when the probability exceeded the threshold. To prevent noisy data from corrupting the model and triggering false positives, we used a Kalman filter (KF) to smooth the power before feeding it into the model. The KF estimated both power and the rate of change in power. The early model used the last 12 hours of data for training. The overhead was a modest 2% CPU usage. The figure below shows an example decision tree in our RF.

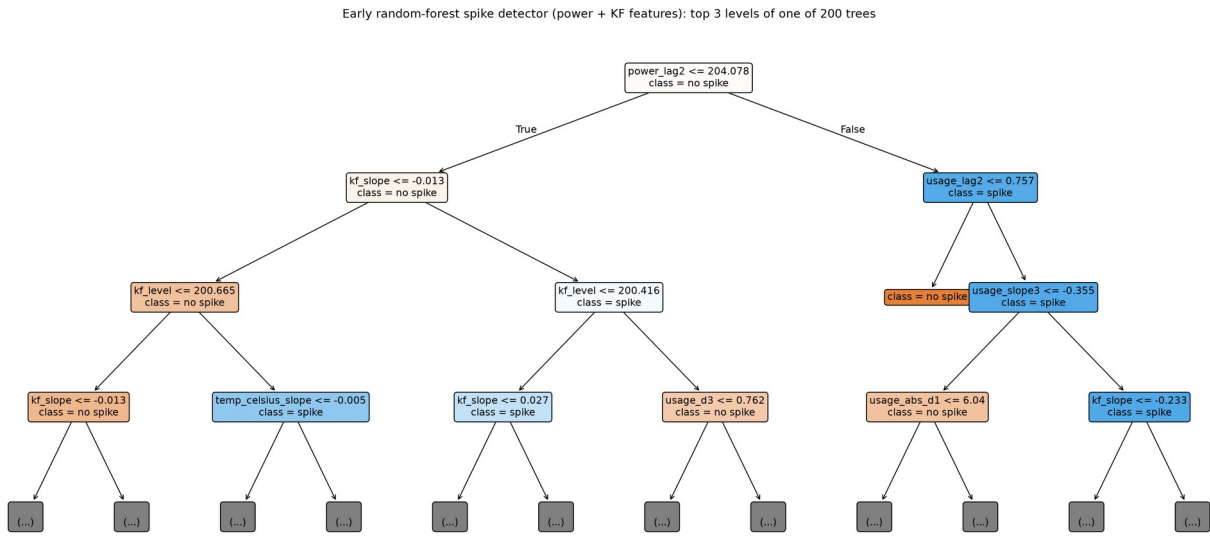


Figure 4.

However, we found that no amount of training could allow our model to forecast spikes in advance. The RF could only identify spikes as they occurred. The Nyquist floor caused 1 Hz sampling to have a minimum 2.0 s lag time. Even with 10 Hz, the power jump was faster than the sample interval. This is because we were training on reactionary data instead of the causal, upstream data needed for a pure power prediction.



Figure 5.

We tried to refine our model using a self-correcting scheme, where a second RF would use a 12-hour window and retrain every hour to replace the incumbent if it scored better in PR-AUC and lead time. This brute force method imposed massive overhead of $\sim 255\%$ CPU usage, which created power spikes on Mycroft. These abrupt power swings were fed back into the RF daemon, causing the model to contaminate itself while being very costly! Thus, we abandoned the continually retraining model and the power prediction avenue.

Prediction to Detection

After failing to predict power, we shifted gears to refining a model that could detect power spikes as soon as they began, mitigating swings rather than erasing them. To pursue this path, we (1) reworked our spike definition, (2) included upstream training data, (3) incorporated Hawkes modeling, and (4) ported model training to reduce overhead. We included an updated tree schematic on the next page.

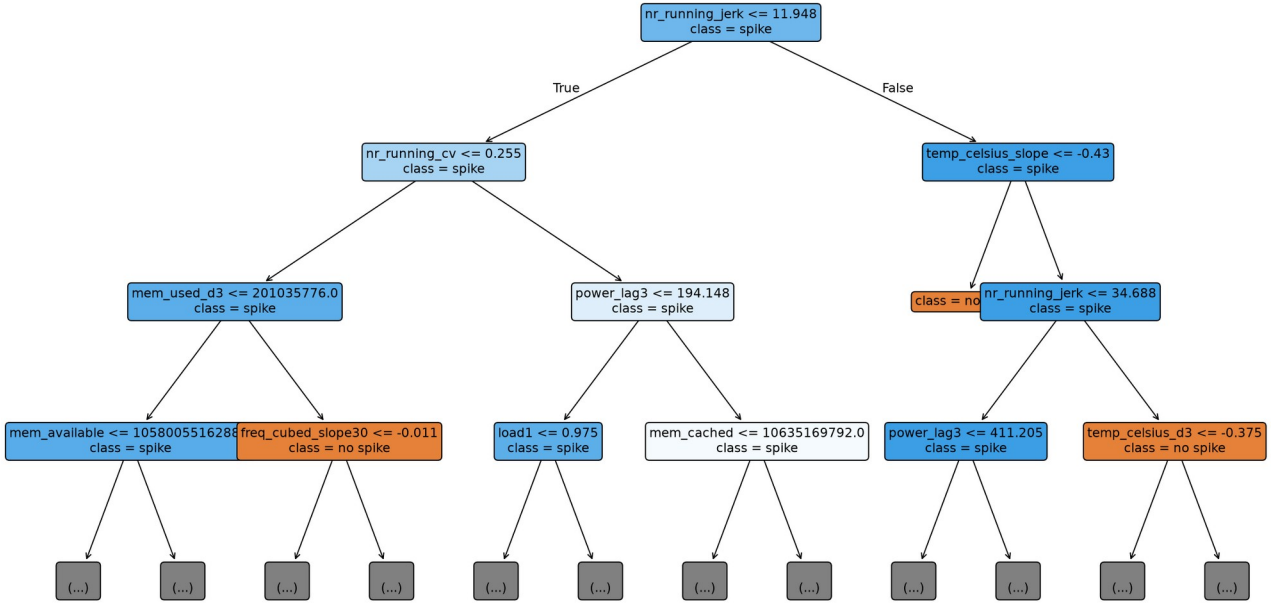


Figure 6.

1. Before, our model defined a spike as the 99th percentile in power change from training data. This was a problem because most of the data came from weekends and non-work hours, leading to a low spike threshold of $|25 \text{ W/s}|$. To avoid choosing an arbitrary power-cutoff rate on a skewed distribution, we used the peaks-over-threshold (POT) method from Extreme Value Theory (EVT). The “severe” tier fit computed a spike on Mycroft as 97 W/s . We kept the legacy EWMA $|z| > 3$ as an alternative definition. Additionally, we restricted the training dataset to work hours (08:00-16:00 on weekdays) to avoid giving too much weight to idle data.

2. One of the main pitfalls of our prediction engine was the absence of upstream training. To identify causes of an abrupt power swing, we screened the Linux scheduler and system telemetry. The data we looked into included run-queue depth, load averages, context-switch rate, memory deltas, and disk and network throughput. The strongest signal was the Linux kernel run-queue jerk, which is the rate of change in the number of runnable processes (`nr_running`). We read this information using `/proc/stat`. With a Cohen’s d value of 1.5, we found that work tends to pile into the queue before a surge in power.

3. Due to the stochastic nature of spikes, we used quantitative finance methods and extended our analogy of consumer behavior to jobs in a queue. To confirm that `nr_running` was a self-exciting process, we treated the run-queue arrivals as a Hawkes process to test whether an event, like adding a job onto the queue, makes similar events more likely in the future. We measured a branching ratio between 0.84 and 0.96 in a 30-40 second kernel as a diagnostic. We added a fixed-kernel Hawkes intensity as a feature. However, it is mostly redundant because of its strong correlation to the run-queue jerk.

4. Finally, to reduce additional power noise on Mycroft, we ported the perpetual RF training to a local laptop. One script handled training off the server on a 24-hour window and synced the model files to Mycroft, where an inference script scored the newest model daily to decide if the challenger would replace the incumbent. This significantly reduced overhead and avoided self-contamination.

Despite all these changes, we were unable to achieve a model that caught more spikes than `usage_edge`, a basic detector that exclusively looked at the CPU usage and its derivative. Additionally, the RF’s 10-20 second lag time was much greater than the 0.1-second response that `usage_edge` achieved. The RF had a much longer flag duty, meaning that when it would detect a spike, it would keep its flag up for longer than needed, which would keep the secondary workload running when it wasn’t needed. Finally, RF had significantly more false alarms. We retired the RF model in favor of `usage_edge`. The table below compares the performance of both detectors.

Table 9. RF vs `usage_edge`

	RF detector (champion)	<code>usage_edge</code>
Training	Supervised: 47 features (power, KF, memory, thermal, run-queue), refit on a rolling window	None. Two hand-set thresholds, no fit
Trigger	$P(\text{spike within 5 s}) > \text{calibrated threshold (0.29)}$	$\text{usage} \geq 20\% \text{ and } (\text{just crossed } 20\% \text{ or rising } \geq 25\%/s)$
Lag time	10–20 s	0.1 s
Onsets caught	—	552 / 557
Flag duty	17.7 %	0.34 %
False alarms	~141 alarms/hr	5.2 alarms/hr
On-box overhead	~2 % of a core (inference); 255 % during on-box refit	Negligible

**Both run on the same HPL workload, `ai_load.sh`*

MATLAB

Early Motivations and Layout

To avoid power swings without investing in hardware for hyperscaled facilities, one must burn power to transition between high- and low-draw states. This means spending more money to prevent massive grid damage. We wanted to analyze this tradeoff and identify the break-even point. Since we only had access to Mycroft, we used MATLAB to extrapolate the behavior of one server across a full power grid of 10,000 servers.

Our simulation relies on the Simulink Specialized Power Systems library to compute both active and reactive power. It uses a fixed hardware floor and load-dependent Power Factor (PF) for each server, a realistic, scaled facility with Uninterruptible Power Supply (UPS) smoothing, and a grid model specified by a voltage, source strength, and stiffness. Assumptions were made from industry

values for facilities, racks, and Mycroft’s Xeon Gold CPUs and HPE ProLiant DL380 specifications. Rationale for each parameter is organized into a chart below.

Table 10. Grid Assumptions

Parameter	Value	Role	Basis
Servers, N	10,000	Extrapolate one server to grid	Realistic size for micro grid
Base server hardware power	300 W	Account for power that servers draw regardless of CPU load	Estimated from DL380 specs for fans, DRAM, drives, NICs, board, conversion loss
Power Usage Effectiveness (PUE)	Idle: 1.60, Full: 1.15	IT-load dependent, variable facility overhead from cooling and power distribution	1.60 is industry average for small load. 1.15 is for full load in modern data centers.
Power factor (PF)	Idle: 0.85, Full: 0.97	Compute reactive power for power supply based on load	Aligns with industry range for power supplies.
UPS smoothing (τ_{ups})	15 s low-pass filter	Absorb high frequency fluctuations like a real grid	Realistic duration for scale according to real world data
phasor mode grid voltage	25 kV, 3 phase	Speeds up simulation time	Reasonable voltage for North American power distribution
Source strength	100 MVA	Quantifies how grid resists changes to load swings	Wanted 10-20 times larger than facility power

Simulink

In our program, we incorporated a low-pass filter to account for UPS power smoothing. Following modern trends, we used a 3-phase grid, meaning the grid delivers alternating current (AC) on three wires offset by 120 degrees in time. To match the industry standard for distribution voltage, our model had 25 kV between wires. At each node in the grid, we simulated a phasor as a complex number, which represents a sinusoid for current and voltage. This was computationally efficient because we solved linear equations instead of differential equations at every time step in the simulation. Finally, we needed grid stiffness, or the source strength to be small enough to react to abrupt facility load changes on a realistic scale.

Simulink Block for Grid

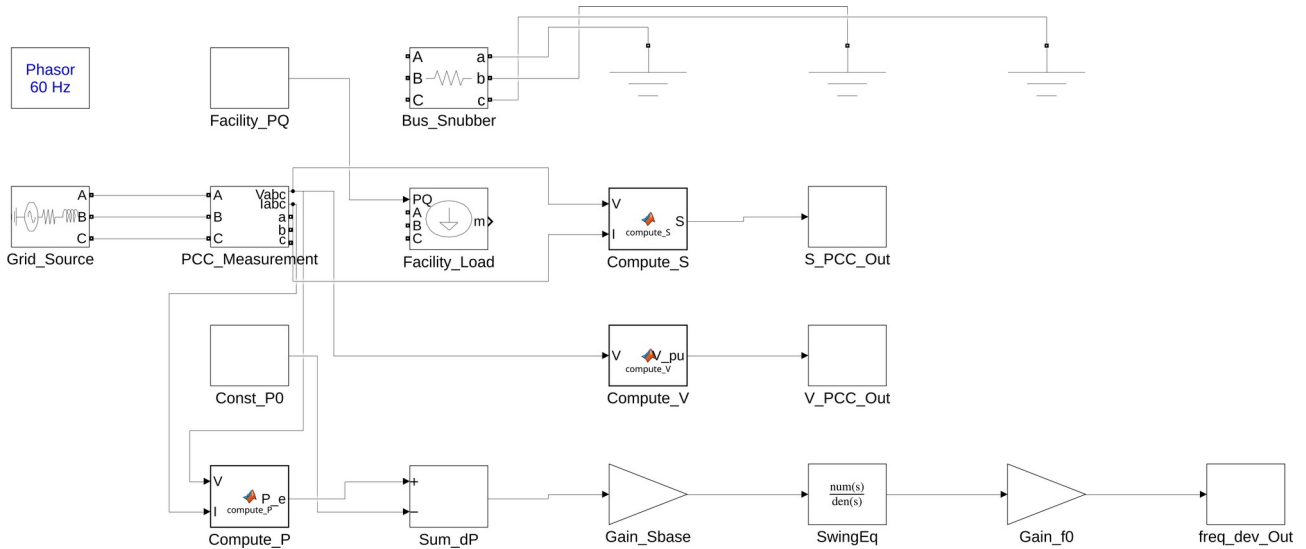


Figure 7.

By modeling facility power as a function of time, we scaled each Mycroft trace (between 111-219 s) to a 10,000-server grid. Simulink calculated grid-related quantities, including dynamic load and complex apparent power, at a point of common coupling (PCC). To emulate a realistic datacenter, we used multiple Mycroft traces with different benchmarks for multiple jobs and applied pseudo-random phase shifts to mimic an unsynchronized load. Later, we simplified our model by accounting for the worst-case scenario where all 10,000 servers ran the same cyclic, largely fluctuating, Linpack workload in sync. This meant the load was linearly scaled by the number of servers. Here is a schematic of our simulation.

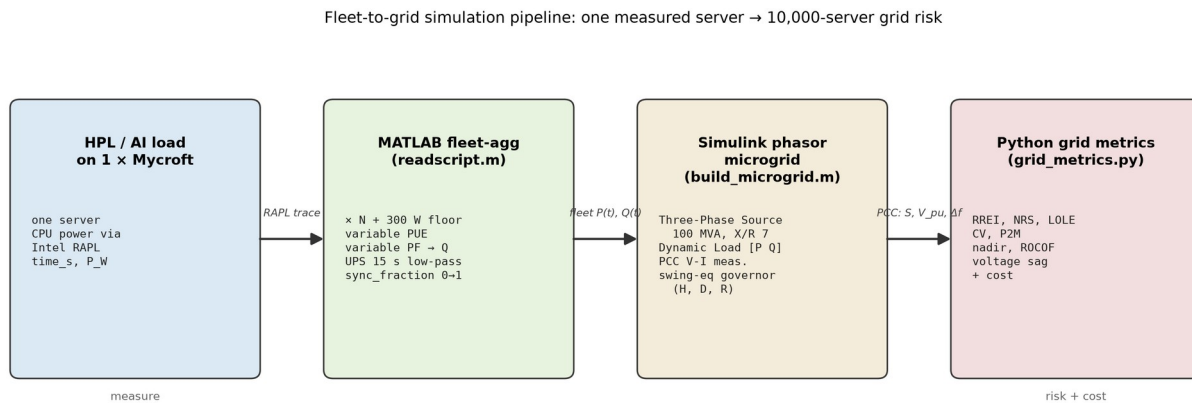


Figure 8.

Metrics

To quantify the benefit of applying a power smoother, we computed the following grid risk metrics after researching safety standards established by the North American Electric Reliability Corporation (NERC) and the Institute of Electrical and Electronics Engineers (IEEE). We defined Ramp Rate Exceedance Index (RREI) and Normalized Ramp Severity (NRS) from a published

ramp limit. Coefficient of Variation (CV) and Peak-to-Mean (P2M) ratio are commonly used in the literature. Frequency nadir, Rate of Change of Frequency (ROCOF), and voltage-sag events are defined and used by NERC and IEEE.

Table 11. Measuring Risk

Metric	What it measures	Threshold / standard
RREI	Annual count of times when facility ramp exceeds the limit	20 MW/min
NRS	For ramp exceedances. Measures how far over, how long	Exceedance defined by 20 MW/min
CV	Overall load volatility (σ/μ)	Lower = flatter load
P2M	Worst case volatility ($\max/\mu$)	Lower = less spiky
Frequency nadir	Lowest AC line frequency after a load swing	59.3 Hz (-0.7 Hz, NERC)
ROCOF	How fast frequency changes	None
Voltage-sag events	How many times PCC went below sag threshold	0.95 per unit (IEEE threshold)

To analyze the cost of implementing a power smoother that deploys a secondary workload, we estimated the cost from the area under the power curve over time. However, our model did not pass sanity checks; even in extreme cases, our grid risk numbers did not respond to CPU throttling. We paused our grid simulations due to these unremarkable results, resuming Simulink after making progress on power smoothing mechanisms.

Solutions to Smoothing Power

Overview

The literature has accepted that preventing abrupt swings in power without additional hardware requires (1) smoothly building up a ramp before heavy computation and (2) deploying a secondary workload to reduce the drop in power when the primary workload is finished. Moreover, one has the option of sacrificing performance by limiting power draw and slowing down the processors, although we note that our goal was to minimize any hit on runtime. We developed three distinct solutions that incorporate some of these features: `ramp.c`, Ramp Down (Power Smoother), and governor.

1. Ramp.c

Our first successful attempt at smoothing power, `ramp.c` linearly builds up and winds down a specified number of cores at a predetermined rate through CPU usage. In practice, we ramped up the power to max, allowed the workload to run, then ramped down after completion. Therefore, we maintained performance and removed almost all power swings by paying a premium in additional

energy cost. Although this luxury tax may not be feasible, ramp.c was intended to serve as a baseline, or an elementary textbook solution. An example power profile is given below.

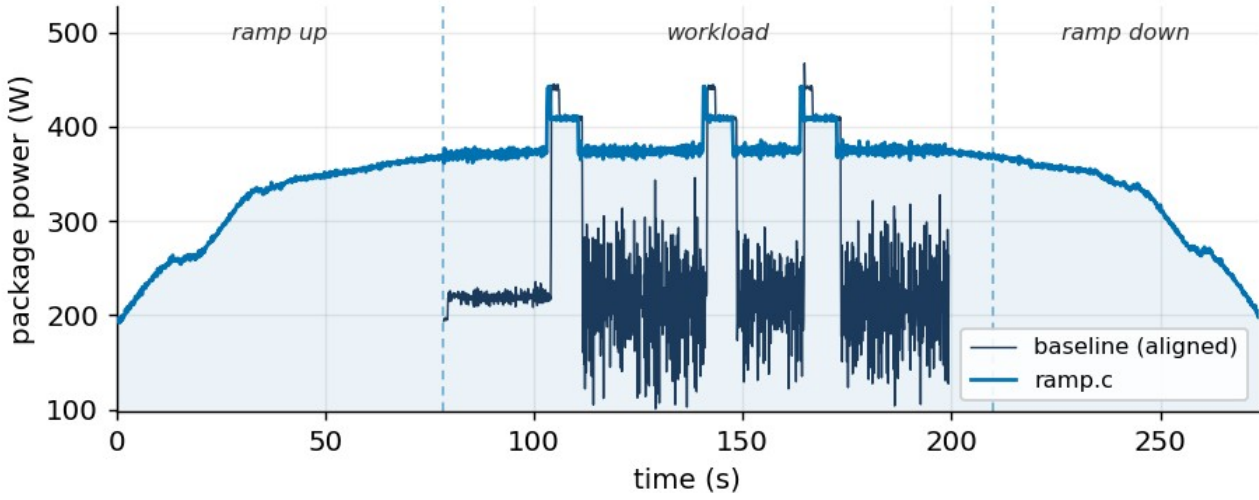


Figure 9.

1.1 composite_ramp.c

Buiding from ramp.c, composite_ramp.c allows for more nuanced features. In this approach, a single manager process (“the manager”) is utilized to control an array of worker processes (“workers”). The workers employ two techniques to manage power fluctuations: filling and transitioning. Filling is performed alongside primary tasks, while transitioning occurs between primary tasks.

1.2 Filling

At its core, filling strives to keep computational resources active throughout the entire duration of a primary task. To accomplish this, the workers are pinned to the same logical CPU cores as the primary task. Whenever the primary task stops utilizing the full extent of its allocated resources, the Linux scheduler engages workers, which perform floating-point arithmetic operations. In order to minimize the performance impact of the workers, they are given the scheduling priority SCHED_IDLE, forcing them to immediately yield CPU time at the request of the primary task.

1.3 Transitioning

Each time the system shifts to a new primary task, several steps are taken to ensure that the power consumption changes gradually. At the beginning of this process, the manager checks a user-defined resource allocation file, classifying the CPU cores according to Table 12.

Table 12. CPU Core Classification

	Not Allocated to Previous Task	Allocated to Previous Task
Not Allocated to Upcoming Task	$C_{uninvolved}$	C_{stop}
Allocated to Upcoming Task	C_{start}	$C_{continue}$

Next, the manager performs $\min(|C_{start}|, |C_{stop}|)$ worker replacements, where $|C|$ denotes the size of set C . In the i th replacement, the worker on $C_{stop}[i]$ is destroyed, while a fully active worker process is created on $C_{start}[i]$. Once the replacements have concluded, the manager performs ramping procedures as needed.

Within the scope of ramping, there are two operations available to the manager: ramping up and ramping down. Fundamentally, both seek to gradually change the CPU utilization of worker processes over time. Ramping up involves increasing worker utilization until the worker is fully active. Conversely, ramping down decreases worker utilization to zero, at which point the worker is destroyed. When multiple workers must be ramped, the procedure is completed for one worker at a time.

To determine whether ramping is necessary, the manager checks whether any cores in C_{stop} or C_{start} were uninvolved with the replacement process. If any such cores exist in C_{stop} , they are ramped down. Similarly, leftover cores in C_{start} are assigned new workers, which are ramped up from complete inactivity.

2. Ramp Down

2.1 Introduction

As stated, the solution that was found to smooth out power spikes is to burn more power. The goal of Ramp Down is to smooth out these power swings within the CPU by executing a dummy workload to gradually decrease the power when a “dangerous” power drop is detected. A “dangerous” swing is a power drop rapid and frequent enough to accelerate hardware degradation, increase average CPU temperature, and disrupt local power grids. Ramp Down was accomplished by analyzing literature, experimenting, and implementing a hybrid solution.

To develop the Ramp Down solution, the group reviewed the literature to determine the most effective workload strategies for power consumption, weighing the trade-offs between running useful versus non-useful background jobs. A useful job can be defined as a job that is lower in priority, a background task, etc. This is an efficient way to schedule the use of power burning, but useful jobs take up memory and storage that are required by primary workloads. If the primary task comes back on, the secondary task needs to be interrupted to prevent power stacking. Power stacking is when two power consuming operations run at full capacity simultaneously, causing total power draw to surge past the limit. For the primary task to run efficiently the secondary task needs to stop, which means the states and progress of the job need to be saved and restored. A non-useful job could be simple arithmetic, something that does not do anything except burn power. This solution avoids the challenges of useful jobs, but it is wasted energy. The group decided that for their current project, using a non-useful job was most reasonable for the program.

2.2 What Didn't Work

Before describing how the smoother operates, it is important to showcase what didn't work throughout the experimentation process.

Firstly, it is important to see how different artificial processes, or dummy workloads, correlated to a specific amount of power. This is crucial for the handoff sequence between the power dropping and the smoother taking over to be as clean and precise as possible. In the earlier versions of the smoother, the handoff sequence was not clean and the dummy processes would overshoot power draw instead of matching the power level where the primary task left off.

The next challenge was defining a “dangerous” power drop. This threshold depends heavily on hardware parameters (i.e., server size, core count, number of servers expected to operate synchronously, etc). The team started by experimenting on their server Mycroft, with a simple benchmark that mimicked an AI workload. The benchmark spikes to peak power and abruptly drops to idle. Through trial and error, the team identified -3000 W/s as a dangerous drop threshold for Mycroft.

To detect this threshold, the team developed Python code to monitor rapid power drops. The polling rate varied depending on the operating state of the daemon: a 2 ms monitoring loop in IDLE, sampling every 50 ms, and worker intensity updates every 250 ms during an active ramp. Monitoring samples every 2 ms over a 50 ms span allows the smoother to swiftly catch power drops within the samples, but the 50 ms interval caused detection lags which triggered power voltage dips and missed spikes. In some trials, the smoother missed the drop entirely; in others, it reacted too late, starting dummy workloads after the primary task had already fallen to idle. Lowering the RAPL drop threshold made the smoother overly sensitive, causing it to misinterpret minor system power fluctuations as dangerous drops and triggering Ramp Down prematurely.

Once these initial handoff and lag issues were resolved, a new problem emerged: after completing a successful ramp down, stopping the dummy workload would sometimes drop power fast enough to re-trigger the smoother itself.

Additionally, an important part touched on earlier was preventing power stacking. In order to prevent power stacking, the smoother has to poll RAPL while Ramp Down is active, and if the power increases instead of decreases, the ramp should execute an interrupt event because the primary task came back on.

In the process of addressing these concerns and the high latency of solely relying on RAPL for power drops, the team explored combining power metrics with CPU utilization. CPU utilization updates in 1-4 ms instantaneously, while RAPL updates about every 1 ms, but it has a 20-50 ms hardware delay. While conducting research, the group found that CPU utilization and power have a linear relationship. Because CPU utilization can be detected faster than power, the group can use utilization to monitor power drops with low latency. To prove this relationship, the team observed live graphs of Mycroft while running benchmarks and analyzed how Power vs. Time and CPU Utilization vs. Time were almost identical. After refining the threshold parameters and introducing new monitoring methods, a successful power smoother was developed.

2.3 The Power Smoother

The goal of the Power Smoother is to detect power drops using CPU utilization alongside RAPL power polling to mitigate power swings. This is accomplished by burning power to smoothly and safely guide the server to idle over 6 seconds or less. The smoother uses specific parameters and

criteria to perform successful ramp downs without introducing harmful thermal spikes, false triggers, or detection lags.

2.4 Core Components

The core components of the smoother are the CPUMonitor, worker processes, baseline estimator, and the context piercing channel.

The CPUMonitor is the hybrid early trip wire and online Ordinary Least Squares (OLS) predictor. The monitor polls `/proc/stat` every 10 ms; because it is a kernel feature there is little to no overhead. In addition, because monitor updates immediately when a process sleeps or dies, the drop in workload can be detected 20-50 ms before Intel RAPL registers it. The online OLS Calibration tracks a 1.5 s rolling window of CPU utilization and raw RAPL wattage data to calculate a live linear model (Equation 1). If the RAPL register becomes stale due to hardware update latency, the system smoothly blends raw hardware readings with the math model to bridge the timing gap.

$$P_{estimated} = P_{idle} + \alpha \times Utilization \quad (W) \quad (1)$$

The next core component is the worker processes, or the dummy workload. The script spawns scaled dummy worker processes to match the server's thread capacity. The math engine executes complex, pure Python floating-point arithmetic to consume raw CPU energy. The intensity of the dummy workers reads shared multi-process float values between 0.0 (idle) and 1.0 (max burn) inside a 10 ms window. The split time between calculating math and sleeping is based on this intensity value. To prevent all the workers from bashing the OS scheduler when Ramp Down is almost done, if the intensity drops below 15% they merge their work windows into fewer cycles and call the OS scheduler to yield.

In addition, to know how much power to inject, the daemon must know what idle currently looks like. To accomplish this, the daemon tracks raw RAPL power readings over a rolling 60-second window and establishes a baseline as the 10th percentile low reading. When a massive workload runs continuously for a long period of time, the baseline calculation can increase. To prevent this from happening, there is a guardrail in place that enforces a strict upper limit clamp to keep the baseline estimation anchored to the true hardware idle.

Regarding the context-piercing channel, if there is an external workload manager or application that knows when the job is about to stop, it can broadcast a signal to the smoother. This signal will bypass thresholds and gradient timing checks to start ramping immediately.

2.5 Startup Calibration

When the smoother initiates, it executes a startup calibration to establish the idle baseline and peak wattage the workers use. To accomplish the idle baseline snapshot the program samples the hardware RAPL power for 1 second while worker intensity is 0.0. Equally, the full burn snapshot is completed by setting the worker intensity to 1.0 for 0.5 seconds and taking another reading. The program then uses Equation 2 to find the worker peak in watts. This value represents the 100% power contribution of all dummy workers combined. Due to power scaling linearly with the duty cycle intensity of pure Python math, any intensity value from 0.0-1.0 maps directly to Equation 3.

$$Worker_{peak} = Worker_{total\ peak} - Baseline_{startup} \quad (W) \quad (2)$$

$$Dummy\ Power = Intensity \times Worker_{peak} \quad (W) \quad (3)$$

2.6 Hybrid Dual-Gate Detection

Another important part of the smoother is the Hybrid Dual-Gate Detection Architecture. The gate detection attempts to eliminate false positives without losing speed through a two-tier verification before engaging workers. Gate A is the CPU utilization Early-Tripwire, and Gate B is the immediate RAPL confirmation.

Explicitly, Gate A monitors while the daemon is idle and evaluates a rolling 120 ms window of CPU utilization data. The thresholds for Gate A include a drop of 65 percentage points or more from an active baseline of at least 80% utilization. If it is triggered, the daemon shifts to a “Potential Cliff” holding state. When in Potential Cliff, Gate B verifies that there is an actual drop in power by speeding up the loop to check RAPL every 2 ms. Within Gate B there is confidence scaling, this is an important check that dynamically adapts the strictness of the Gate based on how strong the evidence was at Gate A. Weak evidence, like a 40% drop, could be a brief thread stall, memory lock contention, or CPU scheduling noise. Conversely, strong evidence means the entire workload exited or went idle, this is usually an 80-100% drop.

2.7 Sliding Window RAPL Path

The sliding window RAPL Path acts as a thorough noise resilient analyzer. The goal is to prevent false positives by analyzing a rolling multi sample 300 ms window. The window computes mathematical derivative trends across the entire window to detect a downward trajectory. If a downward trajectory is confirmed it is then passed through the 5 Layer Noise Suppression.

Notably, the 5 Layer Fallbacks and Noise Gating is for when the workload drops in power but not in CPU Utilization. The 5 layers is to prevent false alarms that could be caused by smaller background tasks, OS jitter, etc.

Layer 1: Pre-Drop Evaluation Gate. It verifies that power is running a minimum of 60 Watts above the idle baseline. This is to prevent deep idle fluctuations from starting a ghost ramp.

Layer 2: Quiescent Timeout Gate. This gate suppresses triggers if the system hasn’t actually seen any high power operations within the last 20 seconds.

Layer 3: Rolling Variance Floor. The daemon analyzes the last 40 samples, which is about 1 second, to see if the variance exceeds $0.8\ W^2$. RAPL power calculations naturally fluctuate roughly $\pm 0.5\ W$ to $\pm 0.8\ W$ purely from hardware noise. This is to see if the power variance is below the threshold, if the system is in deep idle, and a sudden power drop is most likely a previous sensor read rather than a true workload exit.

Layer 4: Adaptive Pre-Drop History. This layer confirms that the older half of the active window was running at a significant fraction of the server’s maximum workload envelope.

Layer 5: Drop Floor Gate. The last gate verifies the current drop has actually fallen near the power baseline.

In addition to Fallbacks and noise, a concern from earlier versions were late triggers and how they caused voltage dips. To prevent late triggers a Late Trigger Guard was implemented. The guard is used if the system experiences a slight detection lag and might try to start a ramp after the hardware has already settled back to idle. If the power is within 40 watts of the baseline and quiescent for more than 0.35 s, then the smoother abandons the execution.

2.8 Fast Pass RAPL Detection

It is important to realize that the CPU power can drop without a corresponding drop in CPU utilization. The Fast Pass RAPL Detection is designed to catch severe, single step RAPL power cliffs resulting from this exact scenario. Fast Pass has a shortcut trigger operating on 25 ms RAPL samples, monitoring for a massive, immediate drop ≥ 160 W at a rate of -1800 W/s or faster. This shortcut bypasses the full five layer noise checks; however, before executing Ramp Down, the system confirms that CPU utilization is also declining. If CPU utilization remains high, the transient drop may simply indicate a memory bandwidth stall, requiring no ramp intervention. Conversely, when a system experiences a sudden, massive power drop within a single sample, waiting for a multi-sample verification causes unsafe latency and risks missing the power transient entirely.

To accomplish this rapid detection, the system evaluates the delta between the current RAPL reading and the immediately preceding single sample. Relying on single sample intervals demands a strict, elevated threshold to prevent false triggers, requiring a massive instantaneous power delta. Bypassing the multi-step layers ensures minimal execution overhead for Fast Pass.

2.9 False Trigger Prevention

Also, to prevent false triggers from micro-stalls or noise, the RAPL power drop must cross its calculated threshold and hold below it for a continuous 120 ms sample before the daemon commits to starting a ramp. Another way to prevent false triggers is through the median despiking filter. Without the filter, the power readings on the target server show RAPL hardware jitter. This causes phantom single-sample spikes of ± 150 W, even while completely idle. Instead of reading the instantaneous raw RAPL values, the daemon keeps a rolling median of the last 5 distinct hardware refreshes. This filter removes any false triggers while at idle, but adds about a 50 ms latency before any real power drop fully reflects in the filtered reading. However, the daemon's internal state machine explicitly accommodates for this delay so as not to judge its own delay as a secondary cliff.

Furthermore, there is a utilization veto for Fast Pass and sliding triggers. Before committing to a pending cliff from RAPL drops, either from Fast Pass or a sliding window, the daemon checks the CPU utilization over 12 ms. If the CPU utilization is greater than or equal to 80%, the RAPL drop is vetoed. Also, on a larger core system, cores can remain busy while RAPL drops. This can be due to internal phase boundaries, memory stall troughs, frequency droop, etc. It is important to verify the power is dropping and not dipping for a short period, as to not cause a false trigger.

During Ramp Down, the RAPL sensors read the total server power draw. This includes the background environment and the dummy's active contribution. To avoid self-contamination, the daemon misinterpreting its own dummy processes as a background workload, the program calculates the background environment power dynamically every 50 ms step using Equation 4. The

raw wattage is the current unfiltered RAPL reading. This value is smoothed using an EMA (Exponential Moving Average) filter to separate out hardware measurement noise.

$$Worker_{environment} = Raw_{watts} - (Intensity \times Worker_{peak}) \quad (W) \quad (4)$$

2.10 RampDown

Furthermore, once a cliff is verified, the system flips to RampDown and controls the power delivery step-by-step every 50 ms over a maximum span of 6 seconds. This is accomplished through closed-loop intensity control, and abort and resumption checks.

The closed-loop intensity control program measures the total hardware draw and subtracts the current power contribution of its own dummy workers to calculate the true background environment. The loop uses an EMA smoothing filter. The smoothing filter calculates how much power is missing from its target trajectory and only injects that precise amount of power. It also incorporates an asymmetric slew rate: while decreasing dummy power is instantaneous, increasing power is restricted to maximum step up of 0.15 normalized intensity units per tick (15% duty cycle increase). Instantly shedding load is safe because decreasing worker intensity eliminates the risk of power stacking. In contrast, adding a dummy workload carries the risk of overshooting the server's TDP (Thermal Design Power) if the primary workload returns. To avoid this, the rate is limited by the INTENSITY command to 0.15 units per step (15% step).

The daemon knows the target power trajectory, and instead of linearly decaying the worker intensity blindly, the daemon calculates only the power that is missing between the target trajectory and the observed background environment, as highlighted in Equation 5. After the missing power is calculated the program converts the missing wattage to the worker intensity command, Equation 6.

$$Missing\ Power = Target\ Power - Worker_{environment\ EMA} \quad (W) \quad (5)$$

$$Intensity\ Command = \frac{(Target\ Power - Worker_{environment\ EMA})}{Worker_{peak}} \quad (6)$$

2.11 Abort and Resumption Checks

Additionally the abort and resumption checks handle scenarios where the primary workload suddenly comes back on or shifts phases during a ramp. If so, the daemon has to immediately stand down to prevent power stacking. This is accomplished using three triggers: the Absolute Guardrail Ceiling, Level-Based Resumption Check, and Rate-Based Backoff.

During testing, the system's TDP was measured around 470 watts. To guarantee that the secondary workload never pushed the server past this limit, an Absolute Guardrail Ceiling safety threshold was established. If the total package power draw hits 470 watts, the guardrail triggers an immediate shutdown of all the dummy processes.

For the Level-Based Resumption Check, if the total power reads significantly higher than the closed-loop model predicts for five consecutive steps, a "Ramp Interrupted" event is flagged and the workers are shut down.

Lastly, the Rate-Based Backoff observes if power spikes upwards at a rate exceeding +3000 W/s, if so, a "Ramp Backoff" event is flagged and the workers are killed.

2.12 CoolDown

When a ramp finishes or is aborted, it enters a CoolDown phase. The dummy workers suddenly stop burning power and there is an abrupt drop in synthetic load, causing a power fall-off on the hardware sensors. The median desprike filter and the derivative history windows hold onto elevated samples while transitioning back to idle; this would cause a false trigger because the daemon would read its own worker shutdown as a brand new primary workload cliff. To avoid aborting or re-triggering the loop, the CoolDown state flushes the median and derivative windows, pausing to re-prime the median filter with clean idle samples before the re-arming.

In practice, the time the daemon waits in the CoolDown state before listening for new cliffs depends on how cleanly the previous ramp ended. For a normal, successful ramp down, the cool down period is 2.0 seconds. If the ramp down was a fast interrupt (less than 1.0 second into the ramp,) then the cool down period is reduced to 1.0 second. The fast interrupt means the system was not confused for long and needs to re-arm quickly in order to catch a potential follow up cliff. By contrast, for a slow interrupt (longer than 1.0 second into a ramp), the cool down period extends to 3.0 seconds. Because the ramp dragged on before aborting it means the telemetry was noisy, ambiguous, or both. The daemon will need to take extra time to let the historical baselines settle. Lastly, a backoff event triggers a 1.0 second cool down period.

2.13 Conclusion

The figure below showcases how the RampDown program behaves and the different paths a power or utilization drop can take.

Additionally, running RampDown has specific execution requirements, reading Intel RAPL files like /proc/stat, and MSR/sysfs entries requires root privileges.

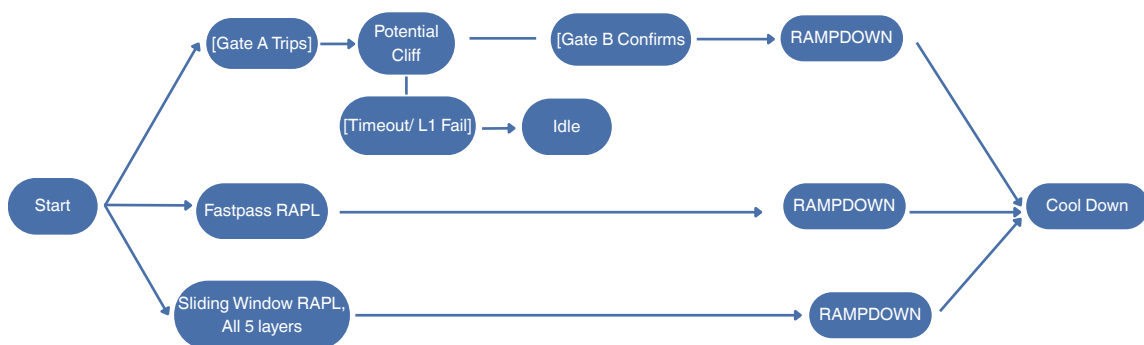


Figure 10.

3. Governor

The two solutions above focus on specific areas for improvement. Ramp.c utterly levels power consumption but requires time to ramp up and down, costing additional time and energy. Ramp Down prioritizes drops in power but can only react to them after the fact. To build a solution that detects both power spikes and drops, the governor was created.

3.1 Detecting: usage_edge

Governor uses `usage_edge.py`, which replaced all previous ML methods for power swing detection. The benefit of using this detector is that it does not rely on power, meaning that unlike Ramp Down, it is not restricted to confirming that the power has changed. It is not necessarily predictive. However, the high correlation between power and usage inspired the `usage_edge` and governor (`usagegov`) stack. We use kernel counters via `/proc/stat` to monitor the usage every 100 ms (10 Hz). Each consecutive pair of samples are computed as a rate. If (1) the interval went from below to above 20% or (2) the rate is above 25% per second, we flag the event as an edge. Depending on whether it is positive or negative, the edge is declared either a rise or a fall. The edge is flagged for 1 second, and the next flag can be fired after 5 seconds to prevent repeated alarms.

3.2 Smoothing: rapl_capper

The governor uses `rapl_capper.py` to act against spikes and drops. As the name suggests, RAPL power caps are employed against spikes. For drops, we use a secondary workload. The program operates in slew mode.

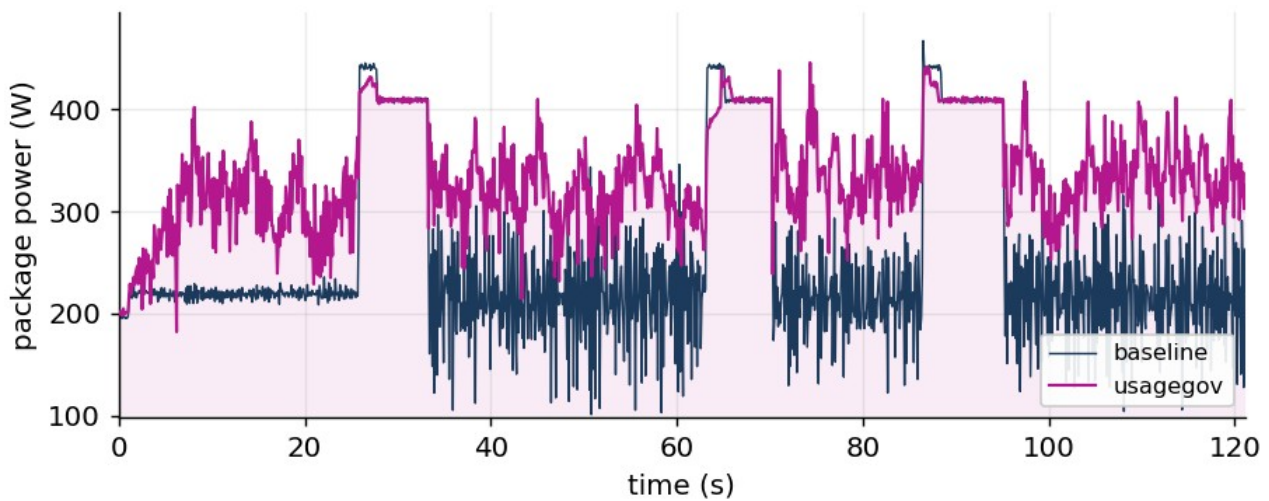
When a rise is detected, we impose a RAPL ceiling 55 W higher than the peak and allow power to climb at a rate that guarantees the ramp up phase is 3 seconds long. This limits impact to the runtime.

Since a power ceiling cannot prevent a fall, we incorporate a ballast technique. Inspired by `composite_ramp.c`, the Linux scheduler assigns jobs from a pool of duty-cycled spinner processes at `SCHED_IDLE` to burn power. All cycles are paused when the primary workload returns.

The governor also has an engage gate, which requires that any smoothing action is only triggered by a transition of $|80|$ W/s or a 45 W/s jump between polls. Having a strict condition for action means high precision and low overhead but vulnerable to inaction. The parameters below describe the governor's settings.

Table 13. usagegov Specifications

Parameter	Value	What it controls
Detector poll interval	100 ms	/proc/stat sampling
Usage threshold	20 %	Edge definition
Usage slope	25% per second	Rise or fall
Flag durations	1 s high, 5 s cooldown	Flag duration and spacing
Control cadence	50 ms	Governor loop
Slew rate	80 W/s	Trigger
Ballast pool	64 cores at 3.26 W/core	Capacity

**Figure 11.**

Experiment Design

Server to MATLAB Pipeline

To analyze the performance and cost of our three solutions to the power swing problem, we designed an experimental procedure. After killing processes on Mycroft to quiesce background noise, we ran three distinct workloads as baselines and compared them to each smoother that ran the corresponding workload. We accomplished this through CSV collection, repeating each run four times. In total, we had one baseline and three solutions for each workload, meaning 12 CSVs across each trial. As mentioned before, our testbed was Mycroft, which uses dual Xeon Gold 6338 CPUs with 128 threads. The idle power is around 200 W and the imposed ceiling is 410 W.



Each cell



Figure 12.

Additionally, we wanted to extend our data analysis to real world, grid level impact. We revisited our Simulink model and corrected it to produce the risk metrics we thought made the most sense. Along with analyzing the Mycroft CSVs to explore server level effects, we passed the same traces through a MATLAB pipeline, including the Simulink grid of 10,000 servers, to extrapolate how our solutions affected grid risk. With this scheme, we scored our solutions on the server and grid level while analyzing the energy and performance cost.

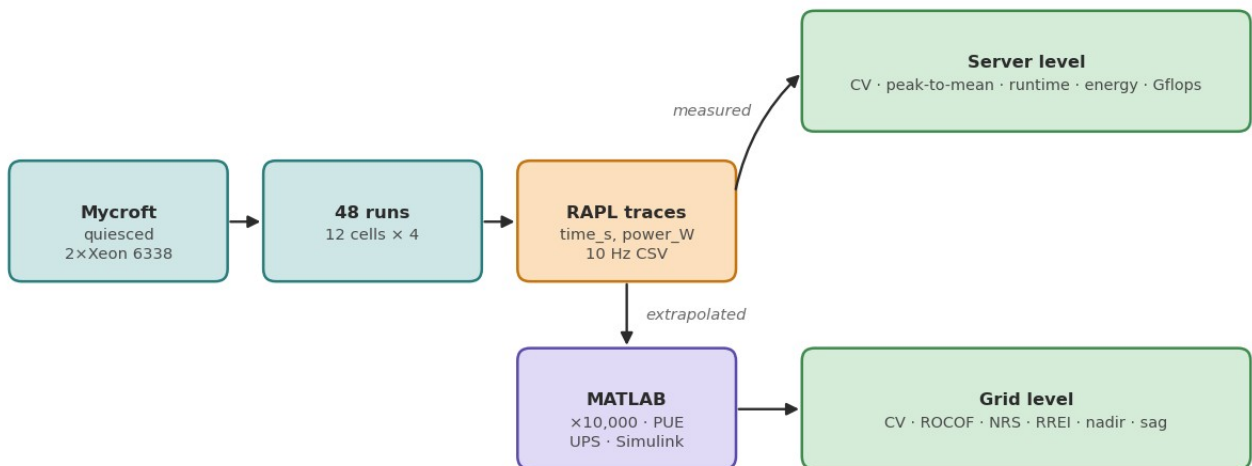


Figure 13.

Workloads

To rigorously test our solutions against a variety of jobs, we designed three workloads with unique power profiles. We employed a mix of oscillations between idle and max power, random peak widths, and random step-like sustained draw. Each workload is explained in more detail with time-series power graphs.

HPL (ai_load.sh)

Our first workload uses HPL to mimic AI training by switching between intensive, compute phases and idle, communication periods. On Mycroft, the baseline workload takes about 18 seconds at high power and 3 seconds at low power. Because the task is cyclical and completion based, we see variable amounts of runtime depending on how our solutions impact the performance. We have used ai_load.sh as a standard, legacy benchmark throughout the summer because of its ability to recreate a simple, realistic HPC job.

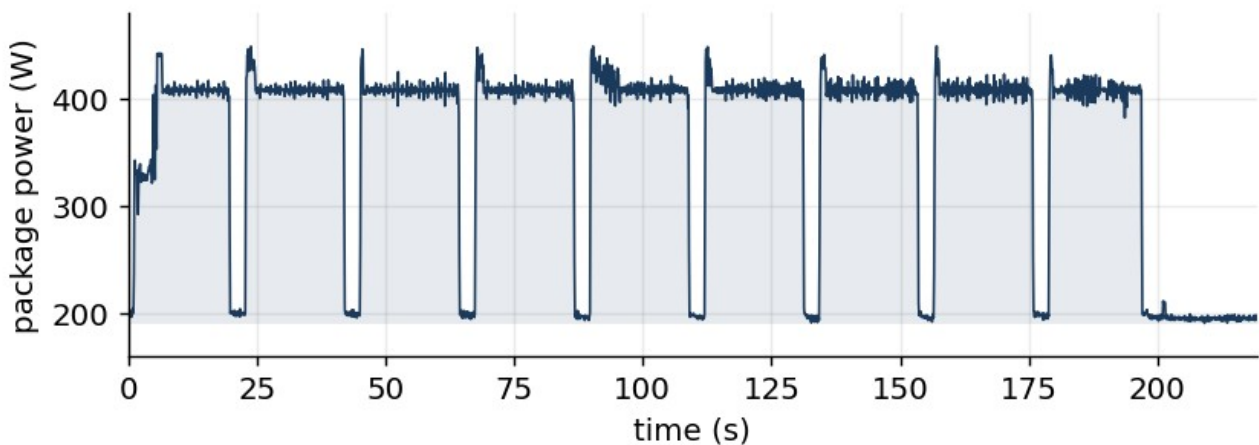


Figure 14.

To execute our HPL workload, we configured the Linpack benchmark to have the following specifications.

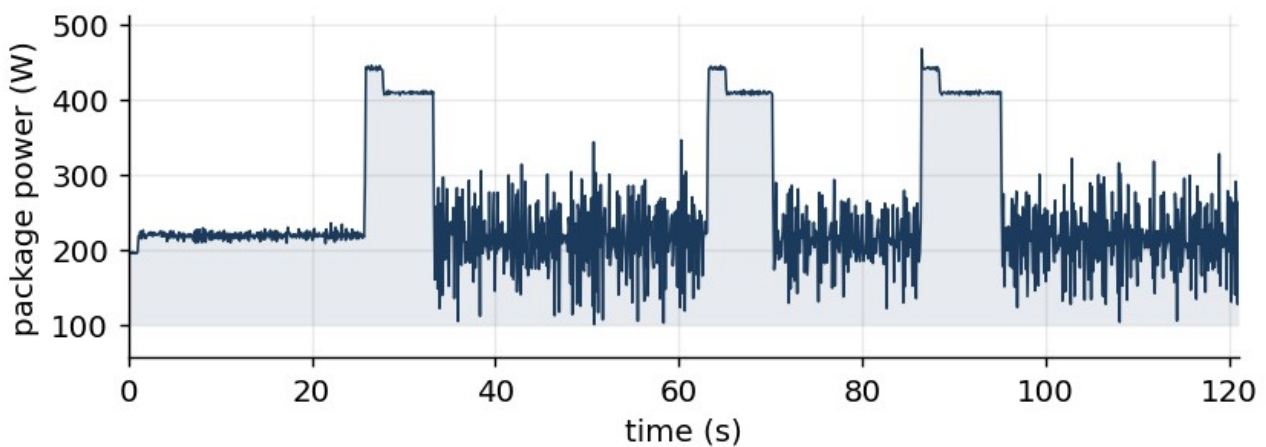
Table 14.

Parameter	Value
Version	HPLinpack 2.3
N (matrix order)	80,000
NB (block size)	192
$P \times Q$	8×16 (128 MPI ranks)
Launch	<code>mpirun -np 128 --oversubscribe ./xhpl</code>
Working set	$80,000^2 \times 8 \text{ B} = 51.2 \text{ GB}$
PFACT / RFACT	Right / Right
NBMIN / NDIV / DEPTH	4 / 2 / 0
BCAST	1ring
SWAP	Mix (threshold 64), L1 and U transposed
Duty cycle (ai_load.sh)	18 s busy, 3 s dip; SIGSTOP/SIGCONT per rank, 3 ms stagger
Residual	1.02×10^{-3} — PASSED

AI Sim 2 Workload

The AI Sim 2 workload was used to train Ramp Down. This is a Python benchmark tool that simulates sharp CPU loads and power consumption of an AI/LLM inference workload. The workload is designed to spike up, stay at a high power for a given duration, and then near instantaneously drop its power consumption.

The benchmark simulates server load across the cores. It begins by spawning worker processes, one worker per logical CPU core. The workers alternate between two main states that are controlled by a shared memory flag. The rest phase occurs when the workers enter a short sleep loop, drawing minimal CPU power. Conversely, the prefill phase is when the workers execute a heavy, continuous loop of dense floating point math. All cores are pegged at 100% utilization to simulate the intense compute phase of an LLM request.

**Figure 15.**

In addition, to keep the workload identical across all trials, an interleaved schedule was used. The benchmark dynamically partitions a total time window of 120 seconds into interleaved rest and prefill segments, featuring three burst events with a minimum duration of 5 seconds each. A random seed is used to allow exact reproduction of the schedule across multiple runs.

To create the abrupt “cliff” power drops, workers check for state updates every 2-5 ms during the prefill phase. As soon as a prefill phase ends, the master process updates the state flag directly. This causes the workers to instantly stop the math execution and drop to idle, producing the steep power drop.

Step Workload

To evaluate the performance of the smoothing software across various conditions, a programmable workload was developed. The behavior of the program is controlled by two parameters: `<thread count>` and `<waveform file>`. When the program begins, it creates the specified number of worker threads, before reading the waveform file.

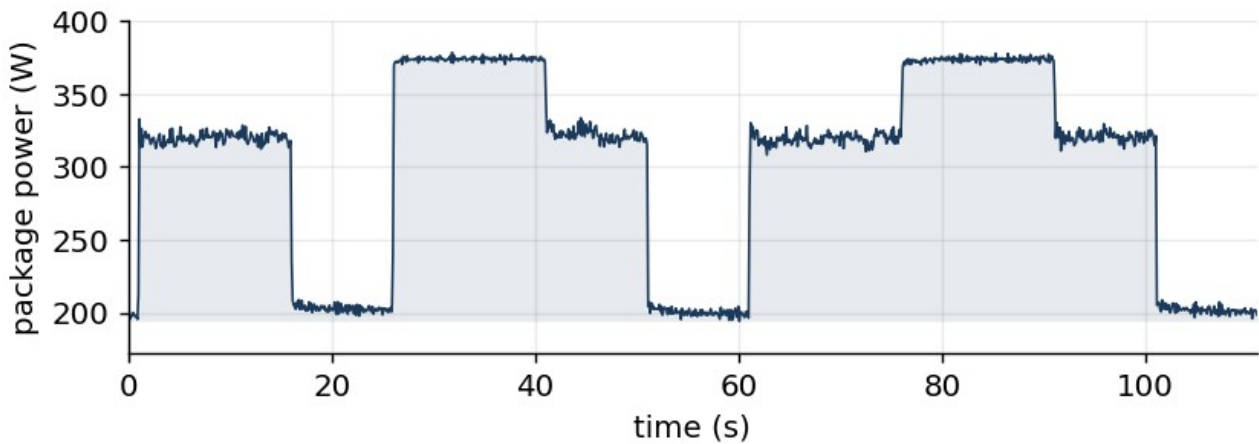


Figure 16.

Inside the waveform file, there is a list of instructions, which are executed in the order they are defined. Each instruction stipulates that worker threads should maintain a particular CPU utilization over a prescribed duration. This functionality is achieved through a work-sleep routine, in which worker threads alternate between arithmetic operations and sleeping.

Altogether, this scheme enables users to construct workloads whose CPU utilization is a specific step function. Since CPU utilization and power consumption are closely correlated, the program is an effective generator of power consumption profiles. Table 15 outlines the particular waveform used for the benchmarking process.

Table 15.

Instruction #	Time (s)	CPU Utilization (%)
1	15	50
2	10	0
3	15	100
4	10	50
5	10	0
6	15	50
7	15	100
8	10	50
9	10	0

Metrics

We note that the server- and grid-level measurements use different standard deviations for power when computing coefficient of variance (CV). While the servers use Mycroft’s raw traces, the grid uses PCC power, which is linearly scaled with variable PUE and passed through a low-pass UPS filter. For ramp.c, we trimmed the Mycroft traces to the window where the workload actually runs. This is to have an even comparison for load volatility. However, we include the up-and-down ramp phases for energy cost and runtime analysis.

On the server level, we scored power smoothing with CV and peak-to-mean. For operational cost and performance tradeoff, we used runtime, energy, and the sustained Gflops for the HPL workload. This is because Linpack is the industry standard benchmark for supercomputers.

Table 16.

Metric	Definition	Units	Better
CV	$\sigma(P) / \mu(P)$ of package power	—	lower
Peak-to-mean	$\max(P) / \mu(P)$	—	lower
Runtime	trace duration, $t_{\text{last}} - t_{\text{first}}$	s	lower
Energy	$\int P \, dt$, trapezoid rule	J	lower
Sustained Gflops	Gflops field of HPL’s WR00R2R4 line	Gflop/s	higher

For the grid, we used the legacy metrics from our previous MATLAB attempt. We listed them again in the table below.

Table 17.

Metric	Definition	Units	Better	Threshold
CV	$\sigma(P_PCC) / \mu(P_PCC)$	—	lower	—
Peak-to-mean	$\max(P_PCC) / \mu(P_PCC)$	—	lower	—
RREI	ramp-rate exceedances: samples with $ dP/dt $ over the limit, annualized	events/yr	lower	0.333 MW/s (20 MW/min)
NRS	$\Sigma(dP/dt - \text{limit}) \cdot dt$ over exceeding samples, annualized	MW/yr	lower	0.333 MW/s
ROCOF	$\max d(\Delta f)/dt $	Hz/s	lower	—
Frequency nadir	$60 + \min(\Delta f)$	Hz	higher	59.3 Hz (NERC BAL-003)
Voltage sag depth	$1 - \min(V)$	pu	lower	—
Voltage sag events	samples below the sag threshold, annualized	events/yr	lower	0.95 pu (IEEE 1159)
Min / mean voltage	$\min(V), \mu(V)$	pu	higher	—
Mean load / peak	$\mu(P_PCC), \max(P_PCC)$	MW	—	—

Results

First Glance

For each solution, we plotted the time-series power data for the smoothed run against its corresponding baseline across all three workloads. We see ramp.c significantly levels power.

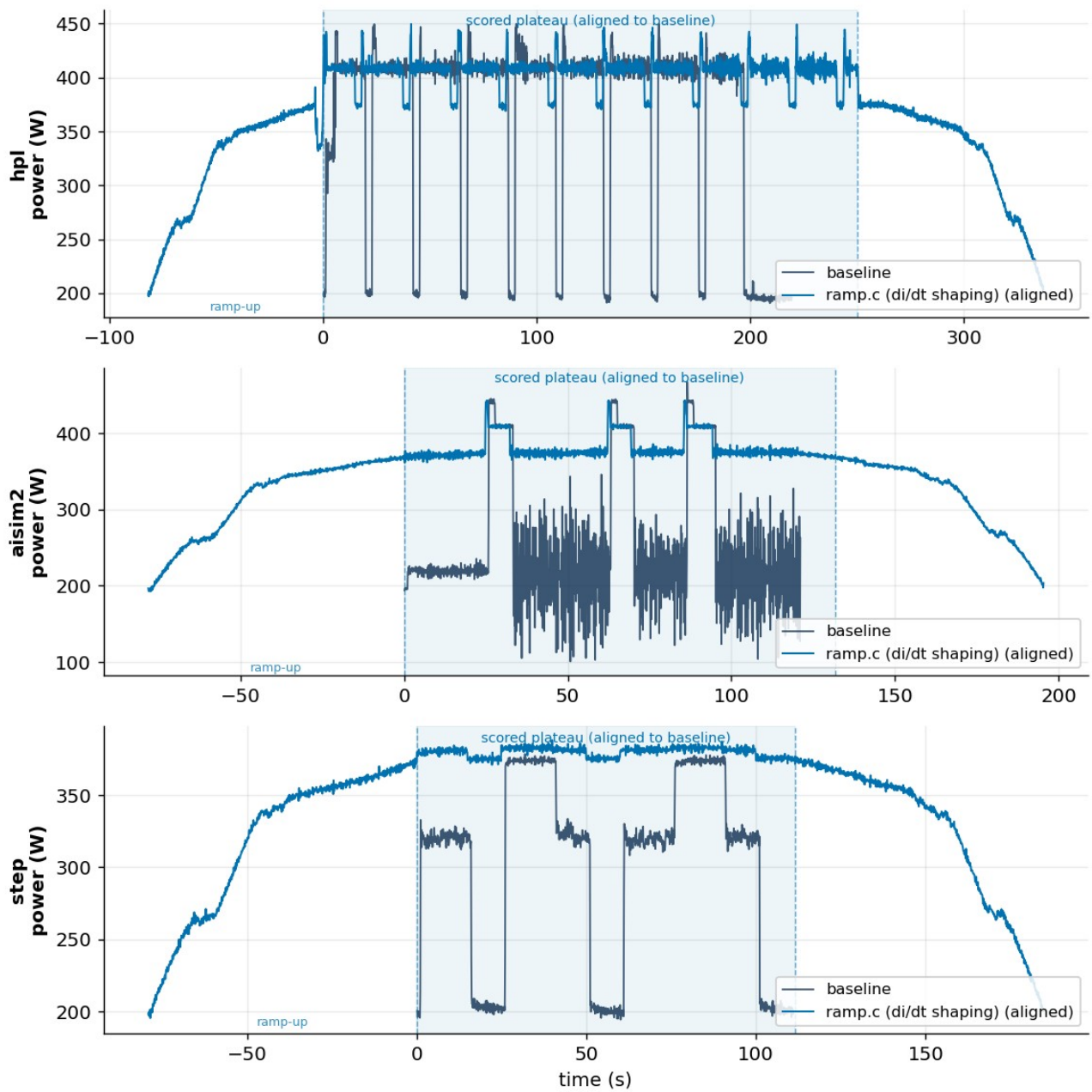


Figure 17.

Ramp down (power smoother) has minimal impact on the power profile. We generally observe noisy data from this solution.

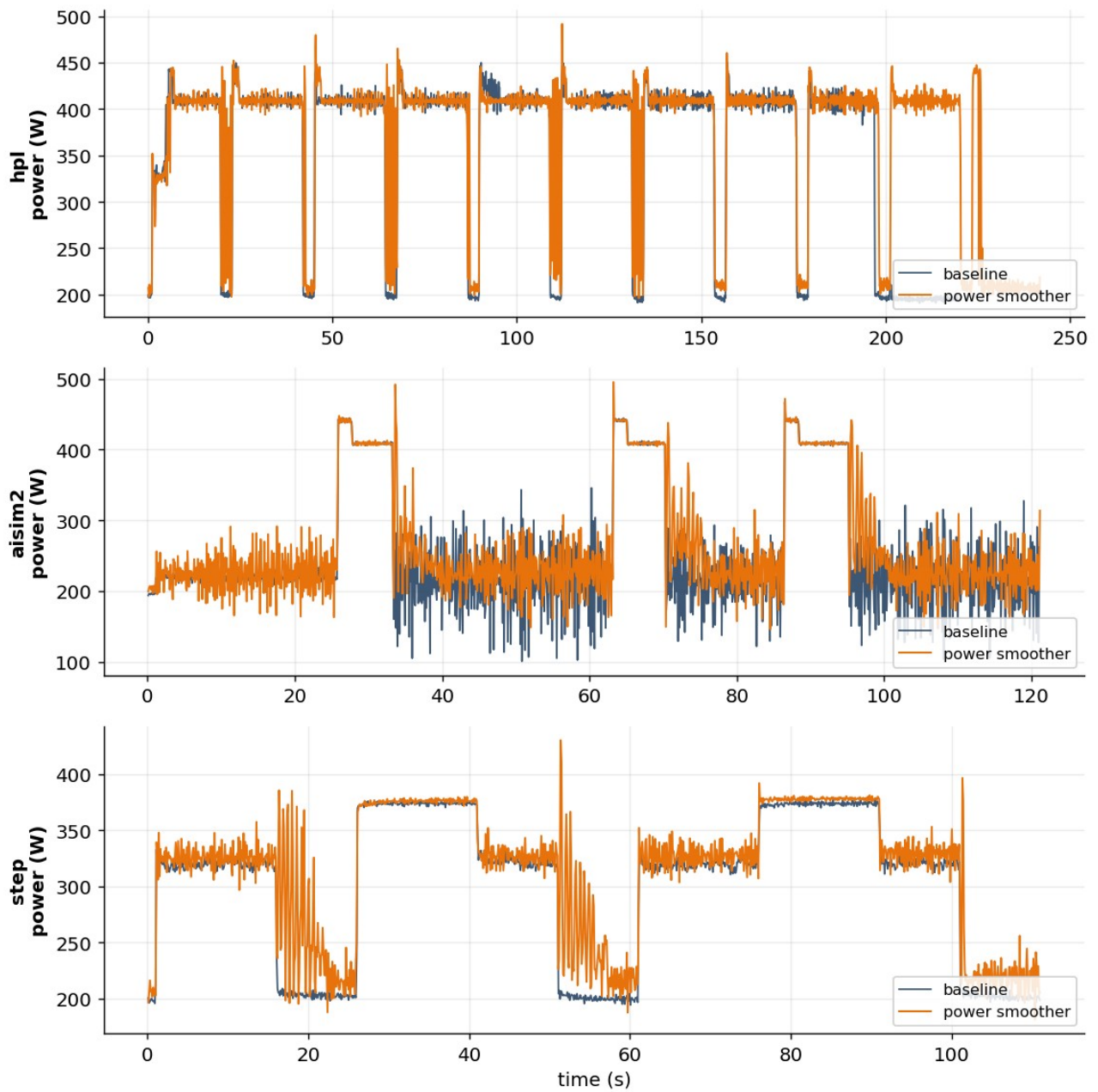


Figure 18.

Finally, the governor (usagegov) applies minimal changes to the power. It seems to marginally slow down upward ramps. The ballast requires more work, as drops persist.

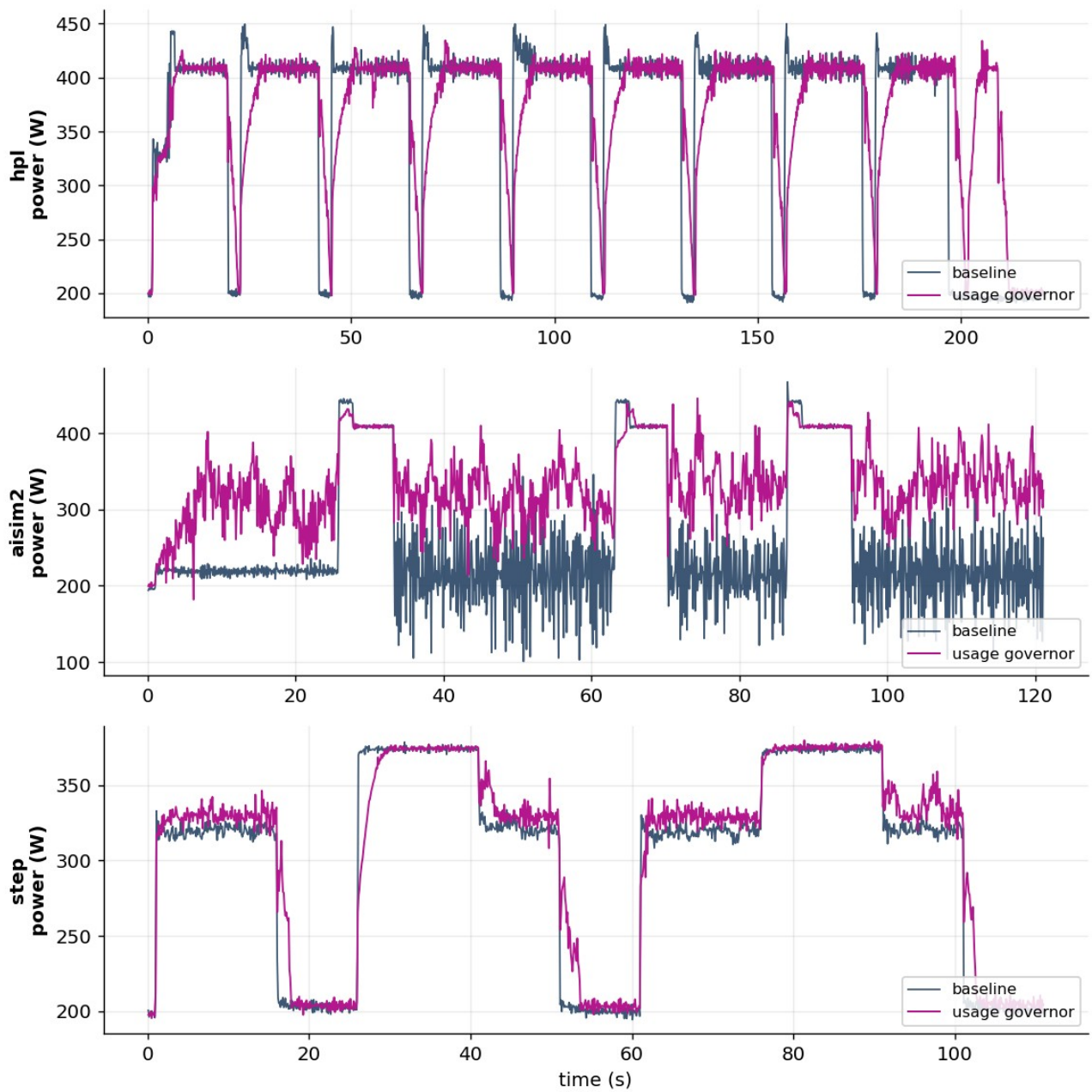


Figure 19.

Server Level Impact

The most illustrative metric in our data was the Coefficient of Variation (CV) in Mycroft's power. Across all workloads, all three of our solutions reduced the CV at the node level, indicating that they reached the server power profile.

Table 18. Mycroft CV for each workload

Solution	hpl	aisim2	step	mean
ramp.c	-38.2 ± 8.3	-57.1 ± 2.2	-36.8 ± 0.5	-44.0 ± 2.7
Power smoother	-15.7 ± 5.7	-11.1 ± 1.2	-12.8 ± 4.6	-13.2 ± 1.8
usagegov	-25.9 ± 7.0	-44.9 ± 7.3	-6.4 ± 0.6	-25.7 ± 4.0

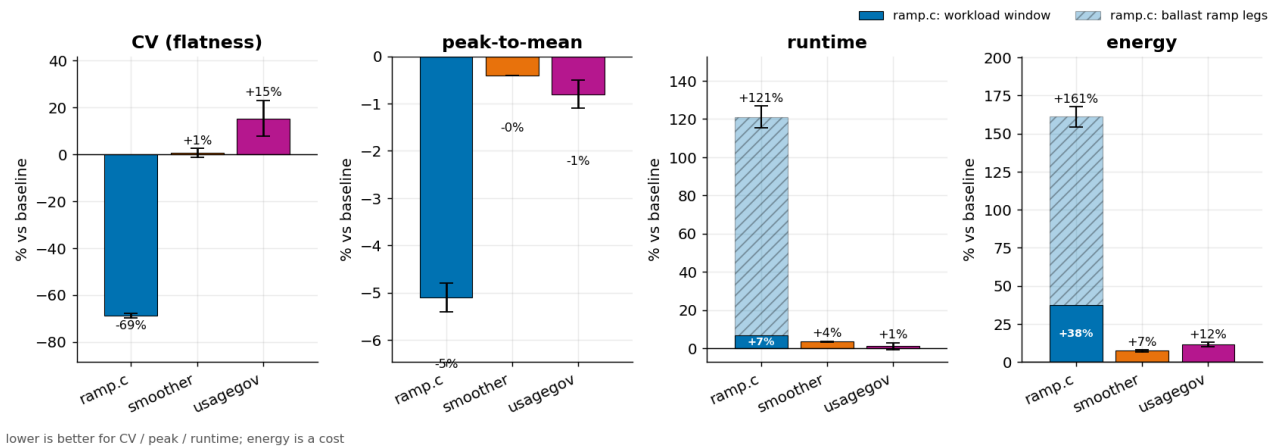
Grid Level Impact

However, our MATLAB simulation indicates that this reduction does not extend the same way to the grid. As we noted in our experiment design, the server- and grid-level pipelines use different time-series power data. For power smoother and governor, we see an increase in CV for the grid level. It is interesting to see that the governor’s CV skyrockets to $+53.6 \pm 19.1$ when using the aisim2 workload. Given aisim2’s noisy time-series power profile, it could be possible that the grid simulation breaks down due to the spin-up phase of the data.

Table 19. Simulink Grid CV for each workload

Solution	hpl	aisim2	step	mean
ramp.c	-76.0 ± 3.2	-64.1 ± 0.8	-66.5 ± 0.4	-68.8 ± 0.9
Power smoother	-3.9 ± 4.9	$+7.0 \pm 3.8$	-1.3 ± 3.9	$+0.6 \pm 1.9$
usagegov	-8.7 ± 6.4	$+53.6 \pm 19.1$	$+1.3 \pm 0.9$	$+15.4 \pm 7.6$

For power smoother and usagegov, the peak-to-mean ratio also does not significantly decrease relative to the baseline. The data shows that ramp.c is the clear winner in reducing grid-level power volatility. It costs an infeasible amount of runtime and energy, whereas the other two smoothers sacrifice small amounts of performance and cost. Grid level results are summarized in the bar charts below.

**Figure 20.**

Cost Analysis

When thinking about cost, we can consider thinking about runtime, energy, and performance in operations per second. We incorporated Gflops measurements for an HPL workload to quantify compute speed. This is because HPL is completion-based while the other two workloads are time-based. The table below shows that power smoother and usagegov imposed small costs while ramp.c had huge hits.

Table 20. Cost of Operation

Metric	ramp.c (with ramps)	ramp.c (workload only)	Power smoother	usagegov
Runtime	+121.1 $\pm$ 5.7%	+6.8 $\pm$ 1.0%	+3.5 $\pm$ 0.2%	+1.1 $\pm$ 1.9%
Energy	+161.0 $\pm$ 6.7%	+37.6 $\pm$ 1.4%	+7.4 $\pm$ 0.6%	+11.6 $\pm$ 1.5%

Table 21. HPL Throughput

Condition	HPL solve (s)	Sustained Gflops	Gflops vs baseline
Baseline	187.1 $\pm$ 1.6	1824.1 $\pm$ 15.2	—
ramp.c	222.9 $\pm$ 20.3	1540.6 $\pm$ 141.3	-15.6%
Power smoother	208.1 $\pm$ 2.3	1640.2 $\pm$ 18.4	-10.1%
usagegov	199.0 $\pm$ 1.0	1715.3 $\pm$ 8.8	-6.0%

Reproducibility of Data

Overall, our four rounds of traces were consistent. We used CV, runtime, and energy for our analysis because they varied the least in our dataset. The graphs below show their repeatability.

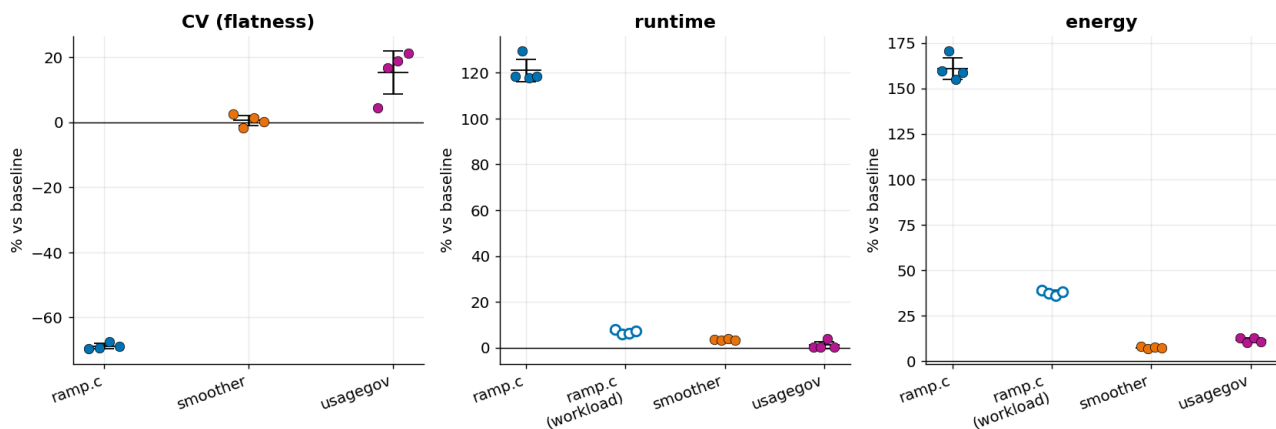


Figure 21.

Complete Dataset

Additionally, we collected other metrics that were less enlightening. We prepared a complete table of all our data from the experiments.

Table 22.

Metric (% change)	ramp.c	Power smoother	usagegov
CV	-68.8 ± 0.9	$+0.6 \pm 1.9$	$+15.4 \pm 7.6$
Peak-to-mean	-5.1 ± 0.3	-0.4 ± 0.0	-0.8 ± 0.3
Runtime	$+121.1 \pm 5.7$	$+3.5 \pm 0.2$	$+1.1 \pm 1.9$
Energy	$+161.0 \pm 6.7$	$+7.4 \pm 0.6$	$+11.6 \pm 1.5$
ROCOF	$+81.3 \pm 2.4$	$+6.0 \pm 1.4$	-1.1 ± 2.3
NRS	$+59.5 \pm 0.8$	$+1.8 \pm 1.3$	-1.9 ± 3.2
Freq nadir	$+75.3 \pm 2.4$	$+5.7 \pm 1.4$	-1.3 ± 2.0
Voltage sag depth	-15.2 ± 1.2	$+2.5 \pm 1.5$	-2.5 ± 2.2
Voltage sag events	-100.0	$+1670.9 \pm 2504.4$	$+1461.0 \pm 2207.6$
RREI	-6.1 ± 0.9	-3.2 ± 0.2	-1.0 ± 1.7

Scoring Detector

Finally, we scored our usage_edge detector against aisim2. Here were our results.

Table 23.

Transition	Detected	Recall	Precision	Median lead
Rise	3/12	25%	16%	-0.22 s
Drop	5/12	42%	23%	-0.37 s
All	8/24	33%	19%	-0.23 s

Although our detector did not score very well, we note that it is a promising start. We believe that with more time and further parameter tuning, we could improve usage_edge. We hope that others can adopt our framework of using various ML techniques, heuristic testing, and statistical analysis to identify triggers and mechanisms for identifying dangerous power swings. Detecting such rare events would greatly enhance the practicality of software-based solutions to this problem.

Conclusion

Overall, ramp.c significantly decreased the magnitude of power fluctuations, but demanded notable sacrifices in performance. Such drawbacks are exacerbated when the workload duration is comparable to that of the transitioning subroutine, which incurs fixed costs in accordance with user specifications. Moreover, the filling mechanism renders ramp.c inherently expensive for workloads that do not utilize a large fraction of their allocated resources.

While power smoother and usagegov had smaller impacts on performance, they did not yield statistically significant benefits with regard to power swings. Ultimately, these shortcomings can largely be attributed to the reactive approach taken by both programs. Since neither employs

corrective measures until after a large fluctuation is detected, they cannot mitigate its initial effects. Furthermore, actions taken to alleviate power swings invariably affect the measurements used for their detection, creating numerous issues that stem from self-reference.

Altogether, one of the three solutions explored proved to be an effective software solution for mitigating swings in power consumption. Due to its negative impact on performance, the software approach is best understood as an alternative, reserved for scenarios where hardware solutions cannot be employed. On a final note, while this study was conducted on CPUs, its overarching conclusions can be extended to GPUs.

AI Acknowledgement

Claude Code was utilized in the production, operation, and evaluation of software described throughout this paper; with this, some figures and tables were produced using AI-generated programs. Google Gemini and Grammarly provided stylistic recommendations regarding the written content of this paper. Once the written content was finalized, Claude Code standardized the formatting throughout the document.

References

- [1] Choukse, E., Warriar, B., Heath, S., et al. (2025). Power stabilization for AI training datacenters. arXiv. <https://doi.org/10.48550/arxiv.2508.14318>
- [2] REDFISH | DMTF. (n.d.). In www.dmtf.org. Retrieved July 24, 2026, from <https://www.dmtf.org/standards/redfish>
- [3] Dementiev, T. W. R. (n.d.). Intel® Performance Counter Monitor - A Better Way to Measure CPU... In Intel. Retrieved July 24, 2026, from <https://www.intel.com/content/www/us/en/developer/articles/tool/performance-counter-monitor.html>
- [4] Tarara, A. (2023). CPU Utilization - A useful metric? In Green-coding.io. <https://www.green-coding.io/case-studies/cpu-utilization-usefulness/>
- [5] North American Electric Reliability Corporation. (n.d.). Reference document: NERC balancing and frequency control. Retrieved July 24, 2026, from https://www.nerc.com/comm/RSTC_Reliability_Guidelines/Reference_Document_NERC_Balancing_and_Frequency_Control.pdf
- [6] North American Electric Reliability Corporation. (n.d.). Whitepaper: Characteristics and risks of emerging large loads. Retrieved July 24, 2026, from <https://www.nerc.com/globalassets/who-we-are/standing-committees/rstc/whitepaper-characteristics-and-risks-of-emerging-large-loads.pdf>
- [7] North American Electric Reliability Corporation. (n.d.). Incident review: Large load loss. Retrieved July 24, 2026, from https://www.nerc.com/globalassets/our-work/reports/event-reports/incident_review_large_load_loss.pdf
- [8] North American Electric Reliability Corporation. (2024). 2024 long-term reliability assessment (corrected July 2025). https://www.nerc.com/globalassets/our-work/assessments/2024-ltra_corrected_july_2025.pdf
- [9] North American Electric Reliability Corporation. (2025). 2025 state of reliability overview. https://www.nerc.com/globalassets/programs/rapa/pa/nerc_sor_2025_overview.pdf
- [10] North American Electric Reliability Corporation. (2025). Effective load carrying capability report. https://www.nerc.com/globalassets/our-work/reports/special-reports/elcc_report_september_2025.pdf
- [11] Institute of Electrical and Electronics Engineers. (2018). IEEE Std 1547-2018: IEEE standard for interconnection and interoperability of distributed energy resources with associated electric power systems interfaces. Retrieved July 24, 2026, from https://en.wikipedia.org/wiki/IEEE_1547
- [12] Dominion Energy. (2024). DER interconnection parameters manual. <https://www.dominionenergy.com/-/media/content/large-business-services/pdfs/virginia/der-interconnection-parameters-manual.pdf>
- [13] National Renewable Energy Laboratory. (n.d.). Smart inverters applications in power systems (IEEE PES TR67). Retrieved July 24, 2026, from <https://www.nrel.gov/media/docs/libraries/grid/smart-inverters-applications-in-power-systems.pdf>
- [14] Best practices for large load interconnections: A North American perspective on data centers. (2025). arXiv. <https://arxiv.org/pdf/2601.12686>
- [15] GridLab. (2025). Practical guidance and considerations for large load interconnections: Interim report. <https://gridlab.org/wp-content/uploads/2025/03/GridLab-Report-Large-Loads-Interim-Report.pdf>
- [16] Enhancing grid resilience for giga-watt scale data centers using high voltage circuit breaker operated braking resistors. (2024). arXiv. <https://arxiv.org/pdf/2512.21295>
- [17] FERC grapples with surging reliability and interconnection demands from data centers. (2025). Orrick. <https://www.orrick.com/en/Insights/2025/11/FERC-Grapples-With-Surging-Reliability-and-Interconnection-Demands-From-Data-Centers>
- [18] NERC tees up plan to assess grid risks associated with data centers. (n.d.). White & Case. Retrieved July 24, 2026, from <https://www.whitecase.com/insight-alert/nerc-tees-plan-assess-grid-risks-associated-data-centers>

- [19] How ISOs and RTOs are addressing large load growth in 2025. (n.d.). Yes Energy. Retrieved July 24, 2026, from <https://www.yesenergy.com/blog/how-isos-and-rtos-are-addressing-large-load-growth-in-2025>
- [20] Solving interconnection bottlenecks with data center load flexibility. (n.d.). Renewable Energy World. Retrieved July 24, 2026, from <https://www.renewableenergyworld.com/power-grid/solving-interconnection-bottlenecks-with-data-center-load-flexibility/>
- [21] Data centers and the grid: How hyperscale computing is reshaping power infrastructure. (n.d.). Power Magazine. Retrieved July 24, 2026, from <https://www.powermag.com/data-centers-and-the-grid-how-hyperscale-computing-is-reshaping-power-infrastructure/>
- [22] Resilience assessment and planning in power distribution systems. (2023). arXiv. <https://arxiv.org/pdf/2308.07552>
- [23] Department for Energy Security and Net Zero. (2023). Exploring reliability standard metrics in a net zero transition. <https://assets.publishing.service.gov.uk/media/65e3a3323f694514a3035f6e/5-exploring-reliability-standard-metrics-in-net-zero-transition.pdf>
- [24] Frequency instability problems in North American interconnections. (2011). U.S. Department of Energy Office of Scientific and Technical Information. <https://www.osti.gov/servlets/purl/1556900>
- [25] Kundur, P. (1994). Power system stability and control. McGraw-Hill.
- [26] Milano, F., Dörfler, F., Hug, G., Hill, D. J., & Verbič, G. (2018). Foundations and challenges of low-inertia systems. In 2018 Power Systems Computation Conference (PSCC). <https://doi.org/10.23919/PSCC.2018.8450880>
- [27] Tielens, P., & Van Hertem, D. (2016). The relevance of inertia in power systems. Renewable and Sustainable Energy Reviews, 55, 999–1009. <https://doi.org/10.1016/j.rser.2015.11.016>
- [28] Institute of Electrical and Electronics Engineers. (2019). IEEE Std 1159-2019: IEEE recommended practice for monitoring electric power quality. IEEE.